SILABUS

ARSITEKTUR KOMPUTER

1. Evolution of instruction sets
2. Processor architecture
3. System Bus dan I/O System
4. Addressing dan memory system
5. Pipelining & RISC
6. Paralel komputer

TERMINOLOGI DAN LINGKUP

Arsitektur adalah segala hal yang memungkinkan bagi programmer untuk dimanipulasi : Set Instruksi, jumlah bit untuk representasi data, Mekanisme I/O, teknik pengalamatan.

Organisasi adalah bagaimana semua fitur yang ada diimplementasikan : Sinyal kontrol, interface, teknologi memori

Arsitektur Komputer = Arsitektur Set instruksi + Organisasi Mesin + …..

LINGKUP ARSITEKTUR KOMPUTER

TEKNOLOGI :

PERUBAHAN DEFENISI ARSITEKTUR KOMPUTER

- 1950 - 1960: Arsitektur komputer :suatu komputer aritmatik.
- 1970 - pertengahan 1980: Arsitektur komputer :suatu desain set intruksi untuk suatu kompiler tertentu.
- 1990 : Arsitektur komputer :suatu bentuk desain CPU, sistem memori, sistem I/O, Multiprosesor, dan Network komputer.
- 2010 : Arsitektur komputer :suatu sistem yang dapat beradaptasi sendiri, struktur yang dapat mengorganisasikan sendiri, sistem DNA / Komputansi Quantum ?

INTERFACE :

- **Antara Lapisan yang berbeda :**
  1. **Application programming interface (API)** : Set rutin yang digunakan aplikasi untuk meminta layanan service dari level yang paling rendah (mesin) yang dikoordinasikan biasanya oleh sistem operasi. Misalnya untuk menangani maintenance task seperti me-manage file dan menampilkan informasi tertentu.
  2. **Instruction Set Architecture (ISA)** : ... Atribut dari sistem komputasi yang dilihat dari sudut pandang programmer, seperti struktur konseptual, perilaku fungsional yang diterapkan pada organisasi aliran data dan kendali atas desain logika dan implementasi fisik pada mesin. - *Amdahl, Blaaw, and Brooks, 1964*

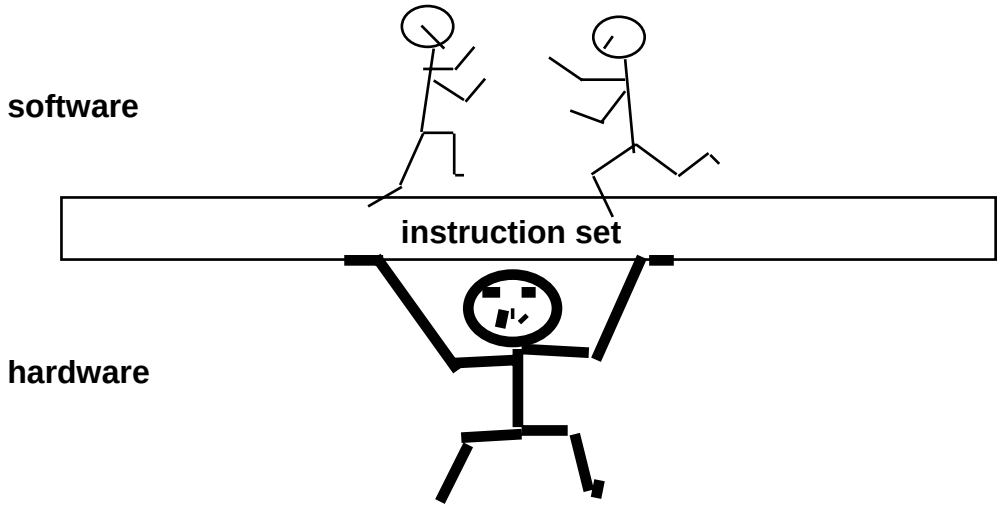
- **Antara modul atau sistem yang berbeda**
  - Protokol Network, Channels/bus

- **Representasi**
  - Character codes (ASCII/EBCDIC/6-bit)
  - Numerics (IEEE 754)

ISA terdiri dari :

1. Organisasi penyimpanan yang dapat diprogram (Programmable Storage)
2. Tipe Data & struktur Data : Encoding & Representasi
3. Set Instruksi
4. Format Instruksi
5. Mode pengalamatan (Addressing) dan mode akses data dan akses intruksi
6. Kondisi-kondisi tertentu

Ilustrasi ISA :



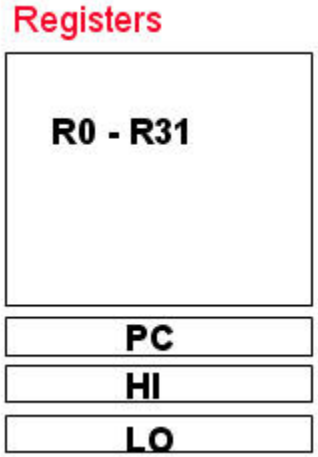
Contoh ISA :

Digital Alpha	(v1, v3)	1992-97
HP PA-RISC	(v1.1, v2.0)	1986-96
Sun Sparc	(v8, v9)	1987-95
SGI MIPS	(MIPS I, II, III, IV, V)	1986-96
Intel	(8086,80286,80386, 80486,Pentium, MMX, ..)	1978-96

Contoh Penerapan ISA :

Pada MIPS R3000 Instruction Set Architecture :

- **Kategori Instruksi**
  - **Load/Store**
  - **Computasional**
  - **Jump dan Branch**
  - **Floating Point**
    - **coprocessor**
  - **Memory Management**
  - **Special**

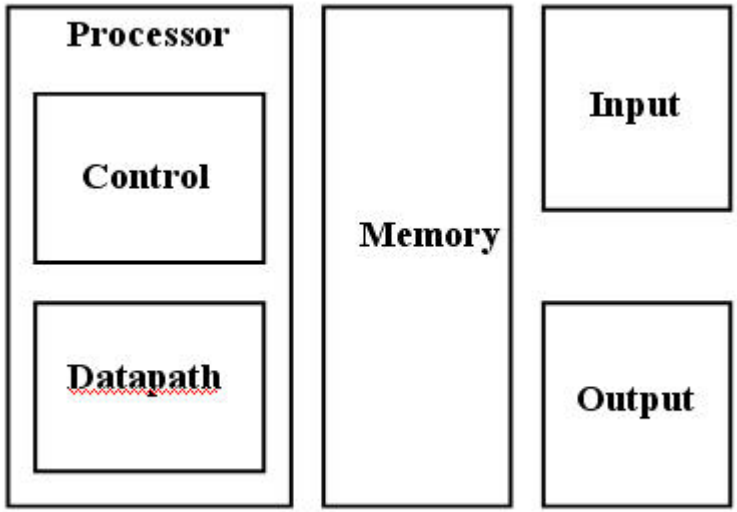


- **3 Format Instruksi 32 bit**

ORGANISASI :

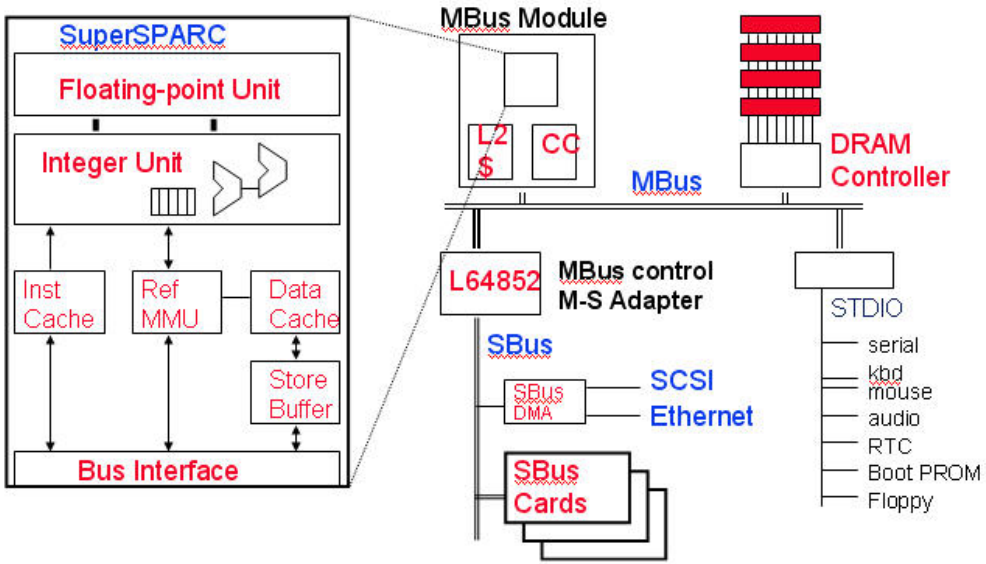
- Karakteristik Kapabilitas & Performance dari masing-masing unit fungsional. Contoh : Registers, ALU, Shift, Logic Unit, ...)
- Bagaimana semua komponen ter-interkoneksi
- Aliran Informasi antara komponen
- Logika pengontrol aliran informasi.
- Suatu kreografi antara Floating ponit Unit dengan ISA
- Deskripsi bahasa Register Transfer Level (RTL)

Gambaran Organisasi :



CONTOH ORGANISASI :

TI SuperSPARCtm TMS390Z50 in Sun SPARCstation20



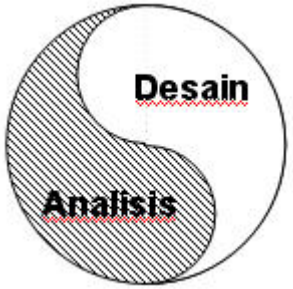
aplikaSI :

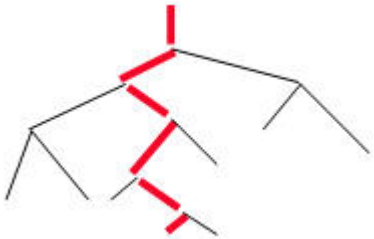
- **CAD, CAM, CAE, . . .**
- **Office (presentasi, spreadsheet, wordprosessor, …)**
- **Multimedia, . . .**
- **The Web, . . .**
- **JAVA, . . .**
- **The Net => ubiquitous computing**

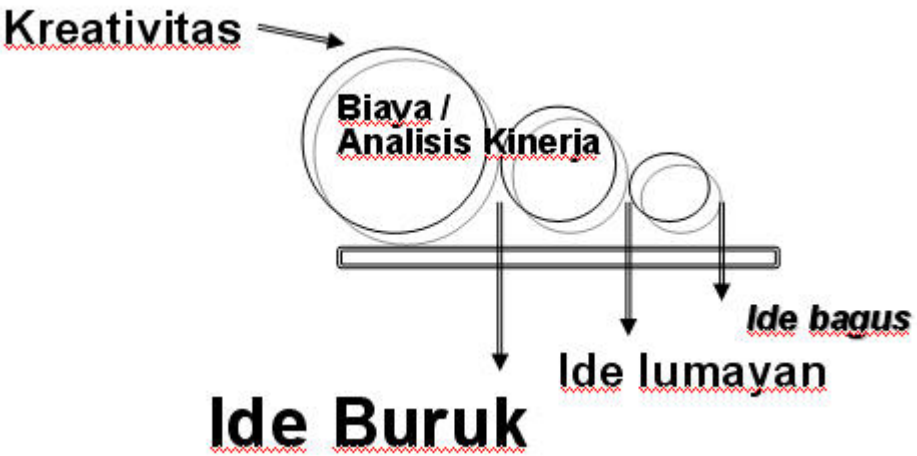
simulaSI & pengukuran kinerja:

Arsitektur adalah proses yang iteratif :

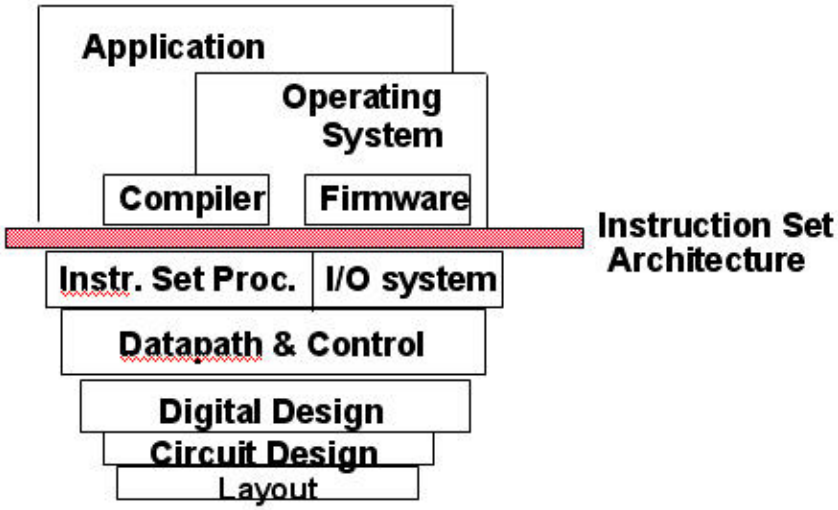
- **Mencari lingkup desain yang mungkin pada semua level sistem komputer**







kesimpulan :
ARSITEKTUR KOMPUTER



LINGKUP ARSITEKTUR KOMPUTER

Instruction Set Design	Computer Organization
Interfaces	Hardware Components
Compiler/System View	Logic Designer's View
"Building Architect"	"Construction Engineer"

SET INSTRUKSI

Dua bagian utama Arsitektur Komputer:
- *Instruction-Set Architecture* (ISA) / Arsitektur Set Instruksi
- *Hardware-System Architecture* (HSA) / Arsitektur Sistem Hardware

- ISA meliputi spesifikasi yang menentukan bagaimana programmer bahasa mesin akan berinteraksi dengan komputer. ISA menentukan sifat komputasional komputer.

- HSA berkaitan dengan subsistem hardware utama komputer (CPU, sistem memori dan IO). HSA mencakup desain logis dan organisasi arus data dari subsistem.

REPRESENTASI DATA

Unit Informasi Dasar dalam sistem komputer :
- satu byte atau 8 bit.
- Word size (ukuran word) – merupakan ukuran register operasionalnya.

Contoh :
1. Komputer 16-bit mempunyai register 16-bit – satu word terdiri dari 2 byte

2. Komputer 32-bit mempunyai register 32-bit – satu word terdiri dari 4 byte.

Representasi karakter yang paling dikenal :
1. Pada PC dan minikomputer adalah *American Standard Code for Information Interchange* (ASCII) : satu byte – satu karakter.
2. Pada mainframe IBM menggunakan *Extended Binary Coded Decimal Interchange Code* (EBCDIC).

Representasi data numerik yang biasa digunakan untuk bilangan bulat dan pecahan (integer dan fraction):
- *Unsigned-binary numbers* (bil. Positif dan Nol)
- *Binary-Coded Decimals* (BCD)
- *Signed-magnitude Integers*
- *Ones-Complement Integers*
- *Twos-Complement Integers*
- *Excess-n*
- *Fraction* (bilangan pecahan)

Representasi *Floating-Point* terdiri dari empat bagian:
- Sign (S)
- Mantissa atau koefisien (M)
- Radix atau base eksponen (R)
- Eksponen (E)

Format *Floating-Point IEEE*
a) *Single Precision* (presisi tunggal) – 32 bit terdiri dari : 1 bit sign, 8 bit eksponen, dan 23 bit mantissa.
b) *Double Precision* (presisi ganda) – 64 bit terdiri dari: 1 bit sign, 11 bit eksponen, dan 52 bit mantissa.

SET REGISTER

Salah satu perbedaan pokok antara satu komputer dengan komputer lainnya adalah pada sifat set registernya.
Set register dapat dibedakan menjadi dua:

- set register tujuan khusus (*special purpose*)
- set register tujuan umum (*general purpose*)

Instruction Register (IR) atau *Program Counter* (PC) – untuk menyimpan alamat instruksi yang sedang dijalankan.

Memory Address Register (MAR) untuk menyimpan alamat memori yang akan diakses.

Memory Buffer Register (MBR) untuk menampung data pada saat operasi pemuatan maupun penyimpanan data.

Indexs Register (IR) untuk menyimpan indeks, misalnya untuk mengakses elemen dalam array.

Flag Register atau Processor-Status Bits, misalnya untuk menyimpan indikasi hasil operasi aritmatika dan logika, seperti:

- P (hasil Positif)
- Z (hasil *Zero*/Nol)
- N (hasil Negatif)
- C (*Carry out*)
- V (*Over Flow*)
- Dan lainnya
-

KARAKTERISTIK DAN FUNGSI SET INSTRUKSI

Operasi dari CPU ditentukan oleh instruksi-instruksi yang dilaksanakan atau dijalankannya. Instruksi ini sering disebut sebagai instruksi mesin (*mechine instructions*) atau instruksi komputer (*computer instructions*).

Defenisi :
Kumpulan dari instruksi-instruksi yang berbeda yang dapat dijalankan oleh CPU disebut <u>set Instruksi</u> (*Instruction Set*).

Elemen-elemen dari instruksi mesin (set instruksi) :

- *Operation Code* (*opcode*) : menentukan operasi yang akan dilaksanakan
- *Source Operand Reference* : merupakan input bagi operasi yang akan dilaksanakan
- *Result Operand Reference* : merupakan hasil dari operasi yang dilaksanakan
- *Next instruction Reference* : memberitahu CPU untuk mengambil (*fetch*) instruksi berikutnya setelah instruksi yang dijalankan selesai.

Lokasi Set Instruksi :
Source dan *result operands* dapat berupa salah satu diantara tiga jenis berikut ini:

1. *Main or Virtual Memory*
2. *CPU Register*
3. *I/O Device*

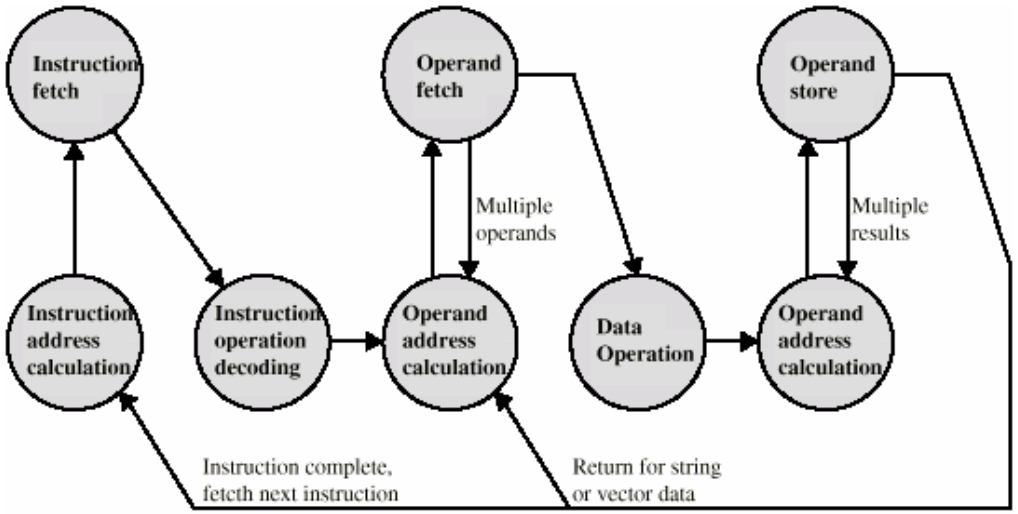
REPRESENTASI INSTRUKSI

Opcodes direpresentasikan dalam bentuk singkatan yang disebut *mnemonics*, yang menunjukkan operasi yang akan dilaksanakan.

Contoh dari *mnemonics* adalah :

- *ADD* *Add (Addition)*
- *SUB* *Subtract*
- *MPY* *Multiply*
- *DIV* *Divide*
- *LOAD* *Load data from memory*
- *STOR* *Store data to memory*
- dsb.

Diagram siklus Instruksi Komputer (Fig. 8-1. Stallings, W. 1990, hal. 293) :



DESAIN SET INSTRUKSI

Desain set instruksi merupakan masalah yang sangat komplek yang melibatkan banyak aspek, diantaranya adalah:

1. Kelengkapan set instruksi
2. Ortogonalitas (sifat independensi instruksi)
3. Kompatibilitas :
   - *source code compatibility*
   - *Object code Compatibility*

Selain ketiga aspek tersebut juga melibatkan hal-hal sebagai berikut :

a. *Operation Repertoire*: Berapa banyak dan operasi apa saja yang disediakan, dan berapa sulit operasinya

b. *Data Types*: tipe/jenis data yang dapat olah

c. *Instruction Format*: panjangnya, banyaknya alamat, dsb.

d. *Register:* Banyaknya register yang dapat digunakan

e. Addressing: Mode pengalamatan untuk operand

FORMAT INSTRUKSI

Suatu instruksi terdiri dari beberapa *field* yang sesuai dengan elemen dalam instruksi tersebut.

Layout dari suatu instruksi sering disebut sebagai Format Instruksi (*Instruction Format*).

Contoh suatu Format Instruksi adalah sbb.:

OPCODE	OPERAND REFERENCE	OPERAND REFERENCE

Ilustrasi Format Instruksi Sederhana
(Fig. 8-2. Stallings, W. 1990, hal. 294)

JENIS INSTRUKSI

1. *Data procecessing: Arithmetic dan Logic Instructions*
2. *Data storage: Memory instructions*
3. *Data Movement: I/O instructions*
4. *Control: Test and branch instructions*

JUMLAH ALAMAT (NUMBER OF ADDRESSES)

Salah satu cara tradisional untuk menggambarkan arsitektur prosessor adalah dengan melihat jumlah alamat yang terkandung dalam setiap instruksinya.

Jumlah alamat maksimum yang mungkin diperlukan dalam sebuah instruksi :

1. *Empat Alamat* (dua operand, satu hasil, satu untuk alamat instruksi berikutnya)
2. *Tiga Alamat* (dua operand, satu hasil)
3. *Dua Alamat* (satu operand merangkap hasil, satunya lagi operand)
4. *Satu Alamat* (menggunakan accumulator untuk menyimpan operand dan hasilnya)

Contoh:

Instruksi untuk menghitung persamaan:

Y = (A – B) / (C + D * E)

a). Three Address Instruction

```
SUB   Y,A,B
MPY   T,D,E
ADD   T,T,C
DIV   Y,Y,T
```

b) Two Address Instruction

```
MOVE   Y,A
SUB    Y,B
MOVE   T,D
MPY    T,E
ADD    T,C
DIV    Y,T
```

C) One Address Instruction

LOAD	D
MPY	E
ADD	C
STOR	Y
LOAD	A
SUB	B
DIV	Y
STOR	Y

JENIS-JENIS OPERAND

- Addresses (akan dibahas pada addressing modes)
- Numbers
- Characters
- Logical Data

Numbers

- Integer or fixed point
- Floating point
- Decimal (BCD)

Characters

- ASCII
- EBCDIC

Logical Data

Bila data berbentuk binary: 0 dan 1

JENIS OPERASI

Data transfer

Transfer data dari suatu lokasi ke lokasi lainnya.
Jika memori dilibatkan:

a) Menentukan alamat memori
b) Melaksanakan transformasi memori virtual-to-actual
c) Menginisiasi Memori Read/Write

Aritmatik

Dapat melibatkan perpindahan data, sebelum atau setelah :

a) Melaksanakan fungsi dalam ALU
b) Set kondisi Kode dan flag

Logikal

Sama dengan Aritmatik

Konversi

Sama dengan Aritmatik dan logical, Dapat melibatkan logika khusus untuk melaksanakan konversi

Transfer Kendali

Update program counter untuk suatu subroutine call/return, me-*manage* parameter *passing* dan *linkage*

I/O

- Mengirim perintah ke Modul I/O.
- Jika memori dipetakan I/O (memory mapped I/O), maka ditentukan alamat memory-mapped-nya terlebih dahulu

Contoh Set Instruksi

Tipe	Nama Operasi	Deskripsi
Data Tranfer	Move (transfer)	Transfer word / blok dari source ke destination
	Store	Transfer word dari processor ke external memory
	Load	Transfer word dari external memory ke processor
	Exchange	Swap content dari source dengan destination
	Clear/reset	Transfer word 0 ke destination
	Set	Transfer word 1 ke destination
	Push	Transfer word dari source ke bagian atas stack
	Pop (pull)	Transfer word dari bagian atas stack ke destination
Arithme-tic	Add	Compute sum of two operands
	Subtract	Compute difference of two operands
	Multiply	Compute product of two operands
	Devide	Compute quotient of two operands
	Absolute	Replace operand by its abs. Value
	Negate	Change sign of operand
	Increment	Add 1 to operand
	Decrement	Substract 1 from operand
Logical	AND	
	OR	Perform the specified logical
	NOT	Operation bitwise
	Exclusive-OR	
	Shift	Left-(right) shift operand introducing constants at end
	Rotate	Left-(right) shift operand around closed path
Transfer Of Control	Jump (Branch)	Unconditional transfer; load PC with specified address
	Jump Conditional	Test specified condition; depending on condition, either load PC with specified address or else Do nothing
	Jump to Subroutine call, Branch and Link	Place current program control information (PC, status register, etc) in known location, e.g. top of stack; jump to specified address
	Return	Replace contents of PC, status register, etc. With information from known location, e.g. from top of stack.
	Execute	Fetch operand from specified loca-tion and execute as instruction; note that PC is not modified.
	Skip	Increment PC to skip the next Instruction
	Skip Conditional	Test specified condition; depen-ding on outcome, either increment PC or else do nothing
	Test	Test specified condition; set flag(s) based on outcome
	Compare	Make logical or arithmatic compa-rison of two or more operands; set flag(s) based on outcome
	Set Control Variabels	Large class of instructions to set controls for protection purposes, interupt handling, timer control, etc.(often privilaged instructions)
	Halt	Stop program execution
	Wait (hold)	Stop program execution; test a specified condition continuously; when the condition is satisfied, resume execution

	No Operation	No operation is performed, but Program execution continues
Input-Output	Input (read)	Transfer data from specified IO port or device to destination, e.g., main memory or processor register
	Output (write)	Transfer data from specified source to IO port or device
	Start IO	Transfer instruction to IOP to Initiate IO operation
	Test IO	Transfer status information from IO system to specified destination
Conver -sion	Translate	Translate values in a section of Memory based on a table of corres -pondences
	Convert	Convert the contents of a word from one form to another (e.g., packed decimal to binary)
System Control	OSCALL	Causes an interrupt, which passes control to operating system

ADDRESSING MODES

Jenis-jenis addressing modes (Teknik Pengalamatan) yang paling umum:

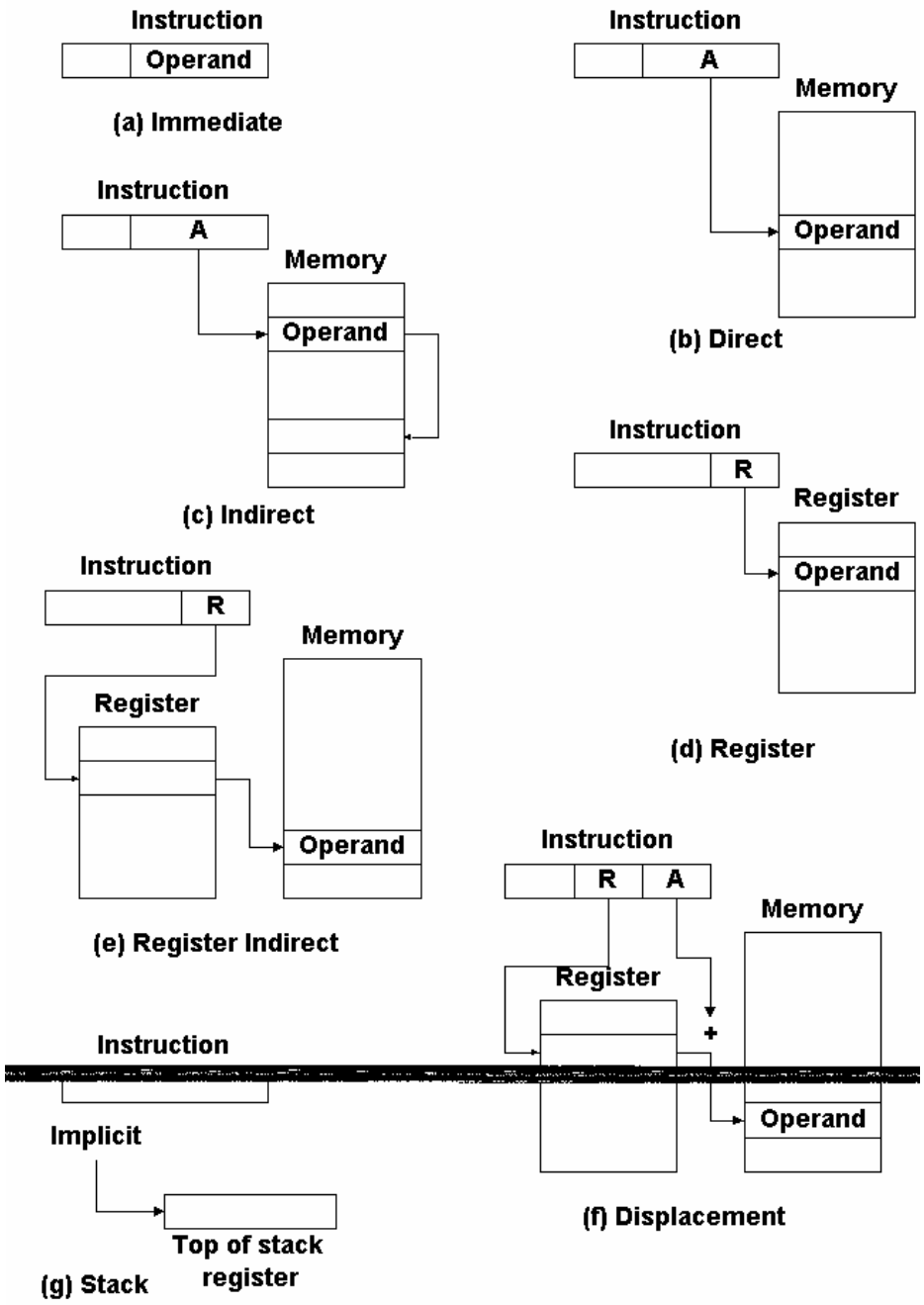
- *Immediate*
- *Direct*
- *Indirect*
- *Register*
- *Register Indirect*
- *Displacement*
- *Stack*

Jenis-jenis addressing modes tersebut diilustrasi-kan pada Tabel dan Gambar . dibawah ini, dengan notasi :

- A = contents of the (an) address field in the instruction
- EA = actual (effective) address of the location containing the referenced operand
- (X) = Contents of location X

Mode Addressing Dasar

Mode	***Algorithm***	***Principal Advantage***	***Principal Disadvantage***
Immediate	Operand = A	No memory reference	Limited operand magnitude
Direct	EA = A	Simple	Limited address space
Indirect	EA = (A)	Large address space	Multiple memory references
Register	EA = R	No memory Reference	Limited address space
Register Indirect	EA = (R)	Large address space	Extra memory reference
Displace-ment	EA=A+(R)	flexibility	Complexity
Stack	EA=top of stack	No memory Reference	Limited applicability

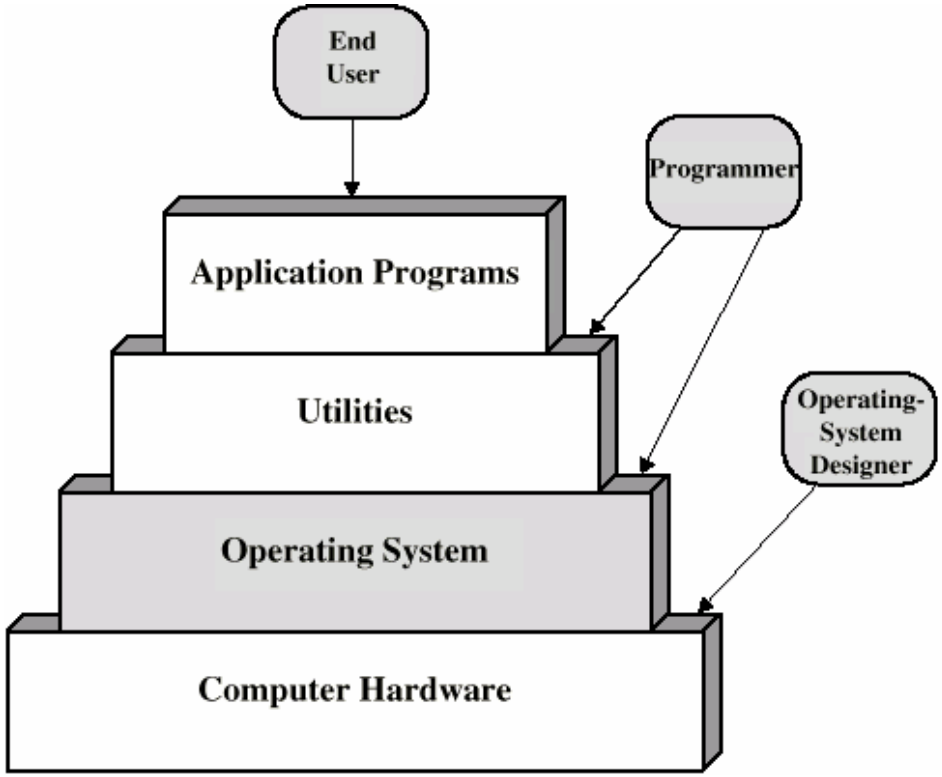


DUKUNGAN SISTEM OPERASI

TUJUAN DAN FUNGSI

Kenyamanan
Membuat komputer lebih mudah penggunaannya.

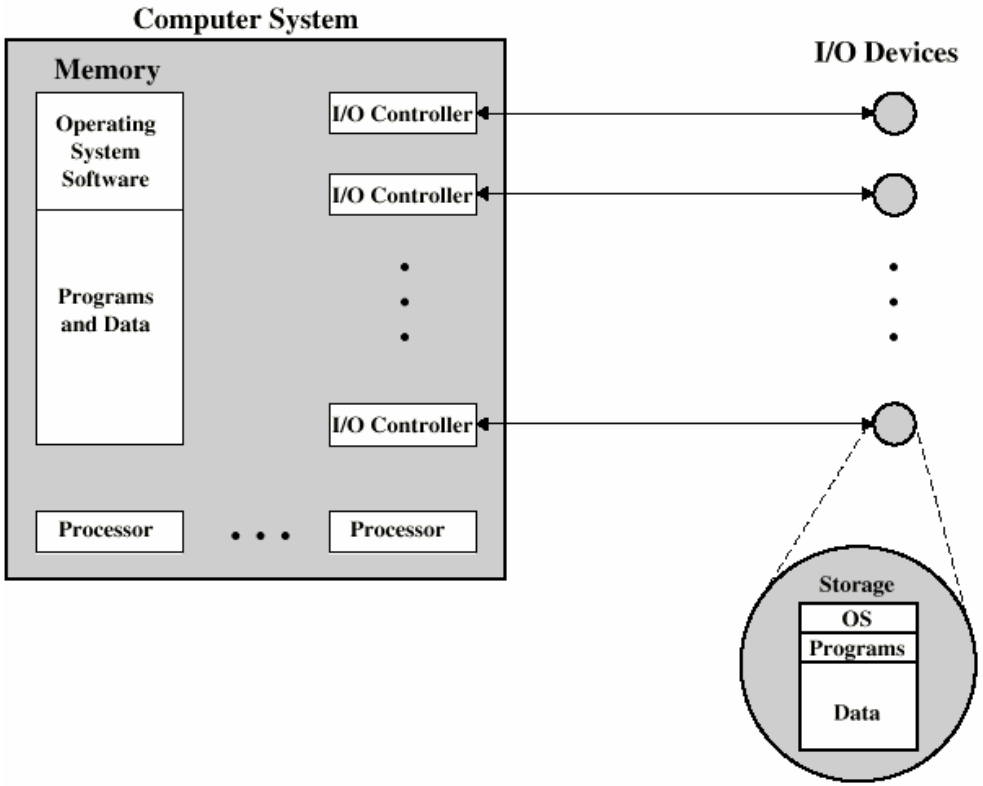
Efisiensi
Penggunaan sumber daya komputer yang lebih baik



Layanan Sistem Operasi :

1. Pembuatan program
2. Pelaksanaan program
3. Mengakses ke device I/O
4. Akses kontrol file
5. Akses sistem
6. Pendeteksian kesalahan dan respon
7. Perhitungan (Accounting)

Sistem Operasi sebagai Manager Sumber Daya :



Tipe Sistem Operasi

- Interaktif
- Batch
- Single program (Uni-programming)
- Multi-programming (Multi-tasking)

Awal Sistem Operasi

Akhir 1940 sampai pertengahan 1950 :

1. Tidak ada Sistem operasi
2. Program saling berhubungan secara langsung dengan perangkat keras
3. Dua permasalahan utama:
   - Penjadwalan
   - Waktu setup

Sistem Batch Sederhana

1. Program monitor yang residen
2. User menngirim job ke operator
3. Operator mem-batch job
4. Monitor control mengendalikan urutan even untuk memproses secara batch
5. Ketika satu job sudah selesai, kendali kembali ke monitor yang akan membaca job berikutnya
6. Monitor menangani skeduling

Bahasa Kendali Job

- Instruksi ke monitor
- Biasanya diawali dengan $ atau beberapa karakter spesial lainnya
- Contoh :
  - $JOB
  - $FTN
  - ... Beberapa instruksi Fortran
  - $LOAD
  - $RUN
  - ... Beberapa data
  - $END
- Bahasa awal (Predecessor) kemudian selanjutnya menjadi scripting languages (Burroughs WFL)

Fasilitas hardware Diinginkan

- Memori terproteksi :
  Untuk melindungi Monitor

- Pengatur waktu (TIMER) :
  Untuk mencegah suatu pekerjaan memonopoli sistem

- Instruksi yang diistimewakan :
  - Yang hanya dieksekusi oleh Monitor
  - Misalnya I/O

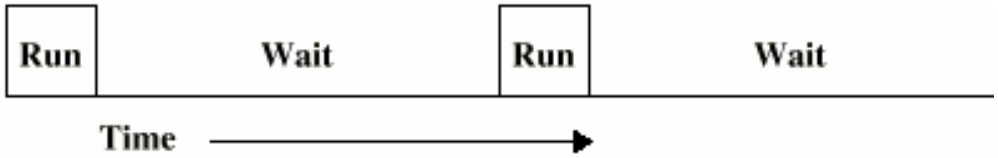
- Interupsi :
  Memungkinkan mempertimbangkan untuk melepaskan (untuk sementara) suatu kendali dan mendapatkannya kembali

Multi-programmed Batch Systems

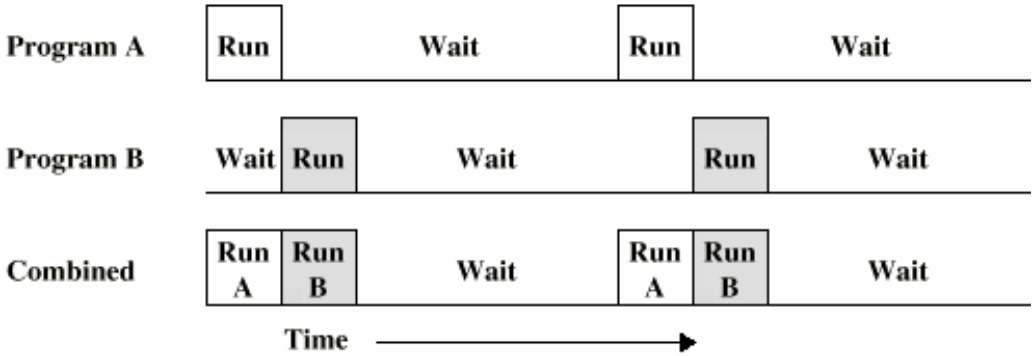
- Device I/O sangat lambat
- Ketika suatu program menunggu I/O selesai bekerja, yang lainnya seharusnya dapat menggunakan CPU.

ILUSTRASI

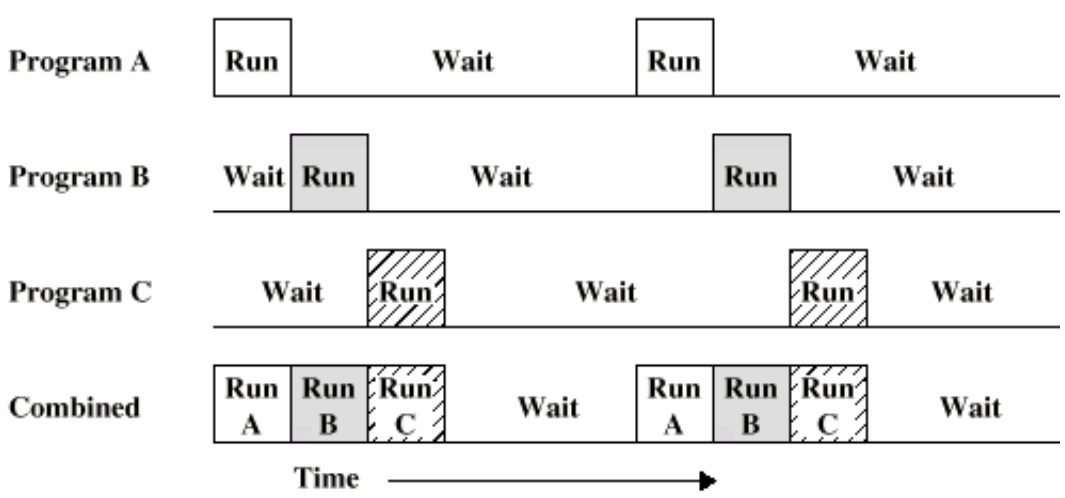
Program Tunggal :



Multi-Programming dengan Dua Program :



Multi-Programming dengan Tiga Program :



Sistem Time Sharing

Mengijinkan para pemakai untuk saling berhubungan secara langsung dengan komputer, yaitu Interaktifitas.

Multi-Programming mengijinkan sejumlah para pemakai untuk saling berhubungan dengan komputer

Penjadwalan (Scheduling)

Merupakan kunci multi-programming, terdiri dari :

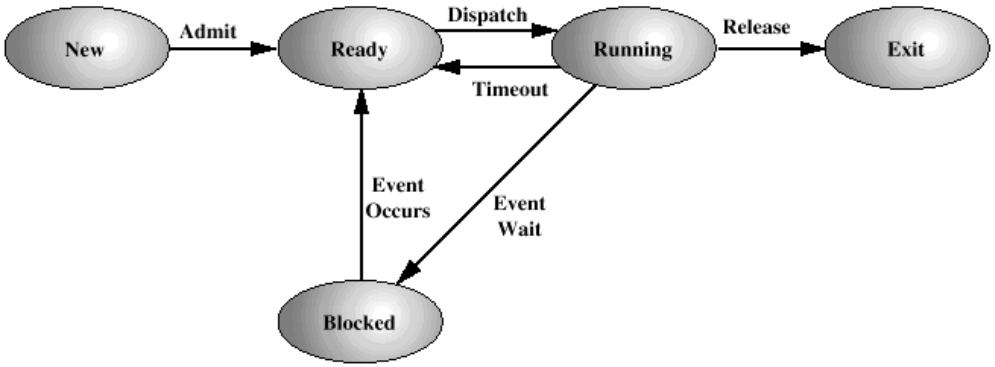
- **Long term**
  - Menentukan program yang mana yang disiapkan untuk memproses. Misalnya mengendalikan derajat tingkat dalam multi-programming
  - Setelah di submit, suatu pekerjaan menjadi suatu proses yang dilaksanakan dalam penjadwalan jangka pendek. (atau menjadi suatu job yang di-swap ke luar untuk penjadwalan jangka menengah)

- **Medium term**
  - Bagian dari fungsi swap.
  - pada umumnya didasarkan pada kebutuhan untuk mengatur multi-programming
  - Jika tidak ada virtual memori, manajemen memori adalah sekedarnya saja.

- **Short term**
  Sebagai DISPATCHER, yaitu pengambil keputusan langsung untuk menentukan pekerjaan mana yang akan dlaksanakan berikutnya. Misalnya : pekerjaan aktual mana yang harus segera masuk ke prosesor pada waktu slot selanjutnya.

- **I/O**
  Hubungan langsung permintaan layanan dari device I/O untuk mendapatkan perhatian Prosesor.

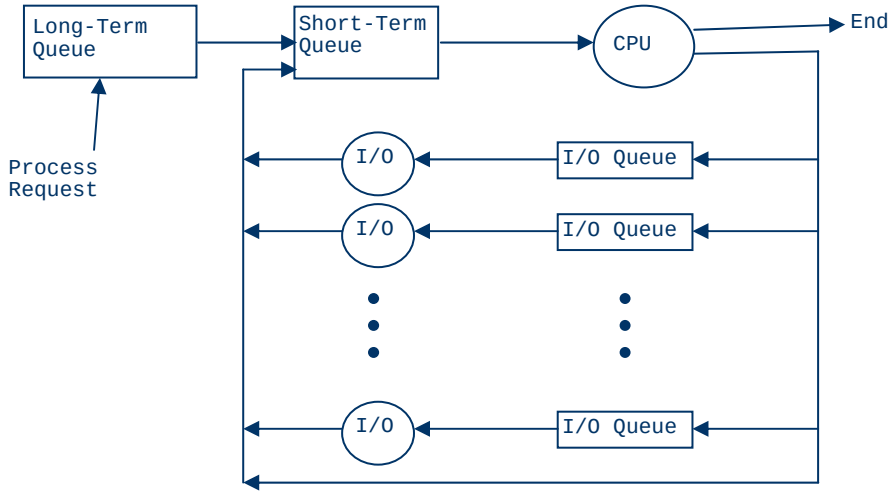
Status Proses



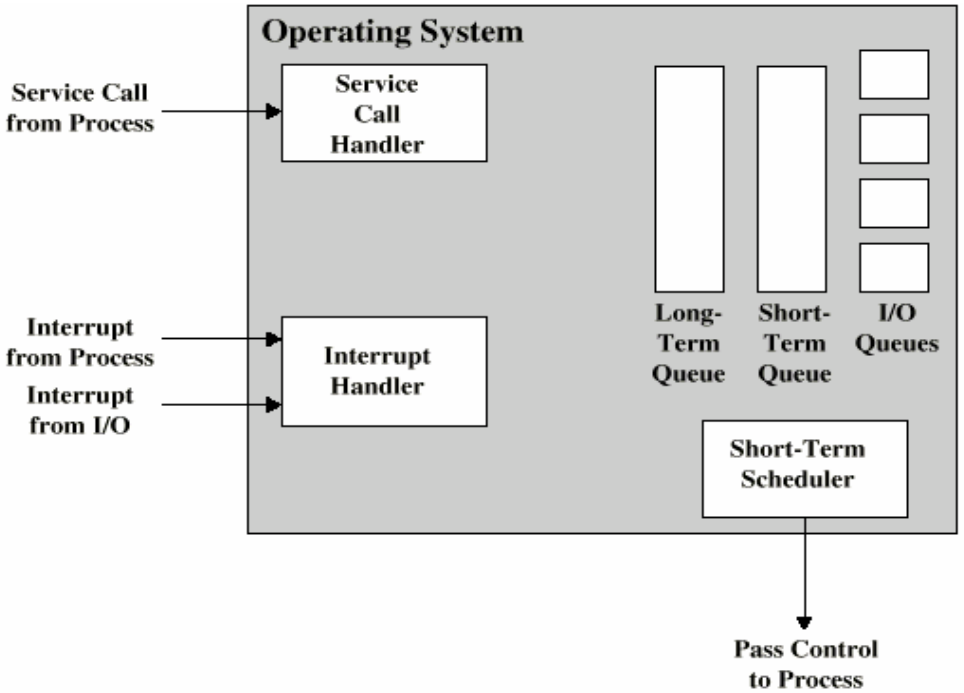
Blok Kendali Proses

- Identifier
- Informasi Status
- Skala Prioritas
- Program counter
- Memory pointer
- Context data
- Status I/O
- Accounting information

Penjadwalan Proses



Elemen Kunci Sistem Operasi



Manajemen Memori

- Uni-Program
  Memori di-split menjadi dua :
  - Satu bagian untuk Sistem operasi (monitor)
  - Satu bagian untuk eksekusi program

- Multi-Program
  User adalah bagian dari sub-divided dan berbagi pakai antar proses aktif saja.

Swapping

Masalah: I/O sangat lambat dibandingkan dengan CPU, bahkan di sistem yang multi-programming, CPU seringkali idle.

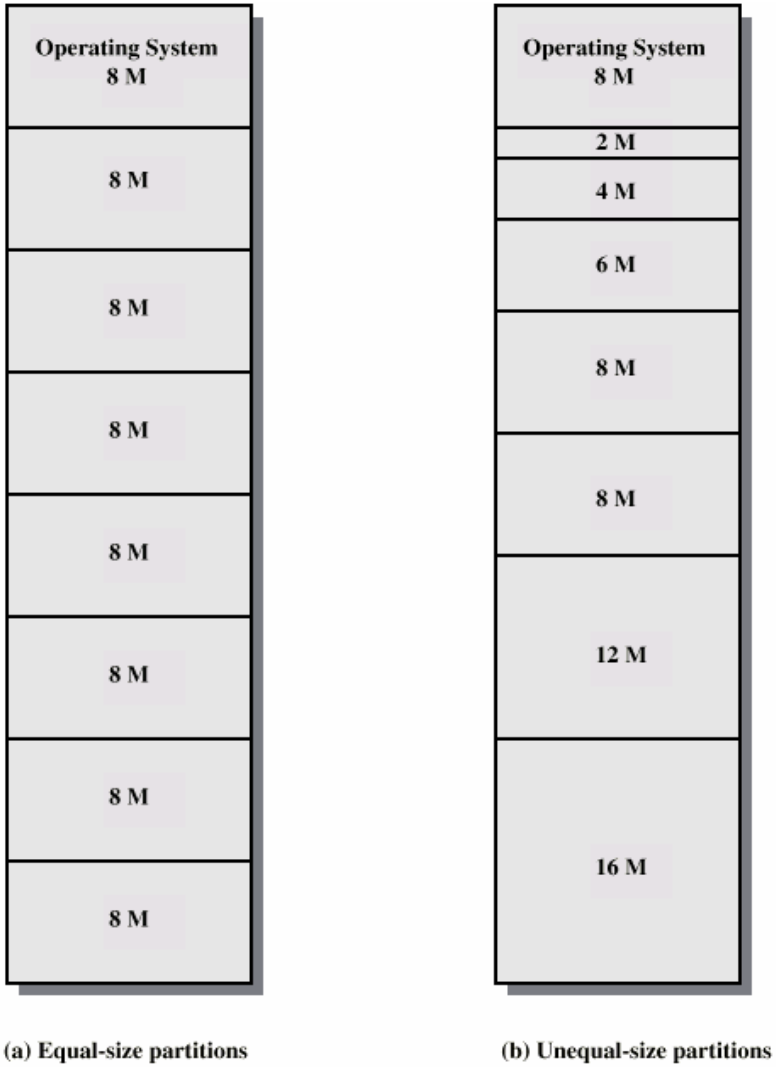
Solusi :
- Meningkatkan memori utama :
  - Mahal
  - Memungkinkan pelaksanaan program besar.
- Swapping
  - Pada Long Term Queue (Antrian jangka panjang), proses disimpan pada disk.
  - Proses “di-swap” ketika ruangan sudah tersedia.
  - Jika proses sudah selesai, akan dipindahkan keluar dari memori utama.
  - Jika tidak satupun dari proses di dalam memori yang siap (misalnya ketika semua I/O di blok) :
    - Terjadi penukaran (swap), proses yang diblok ke antrian yang akan segera dilayani.
    - Terjadi penukaran (swap), proses yang sudah siap atau proses baru.

Partitioning

Defenisi :
Melakukan Split memori menjadi bagian-bagian untuk dialokasikan ke proses (termasuk proses pada Sistem operasi)

Partisi Fixed-sized
- Dapat saja tidak dalam ukuran yang sama.
- Proses dimasukkan ke dalam lubang paling kecil yang paling tepat ukurannya dengan proses tersebut)
- Dapat saja beberapa bagian memori jadi tersia-sia, sehingga membutuhkan partisi dengan ukuran variabel.

Ilustrasi Partisi Fixed-sized

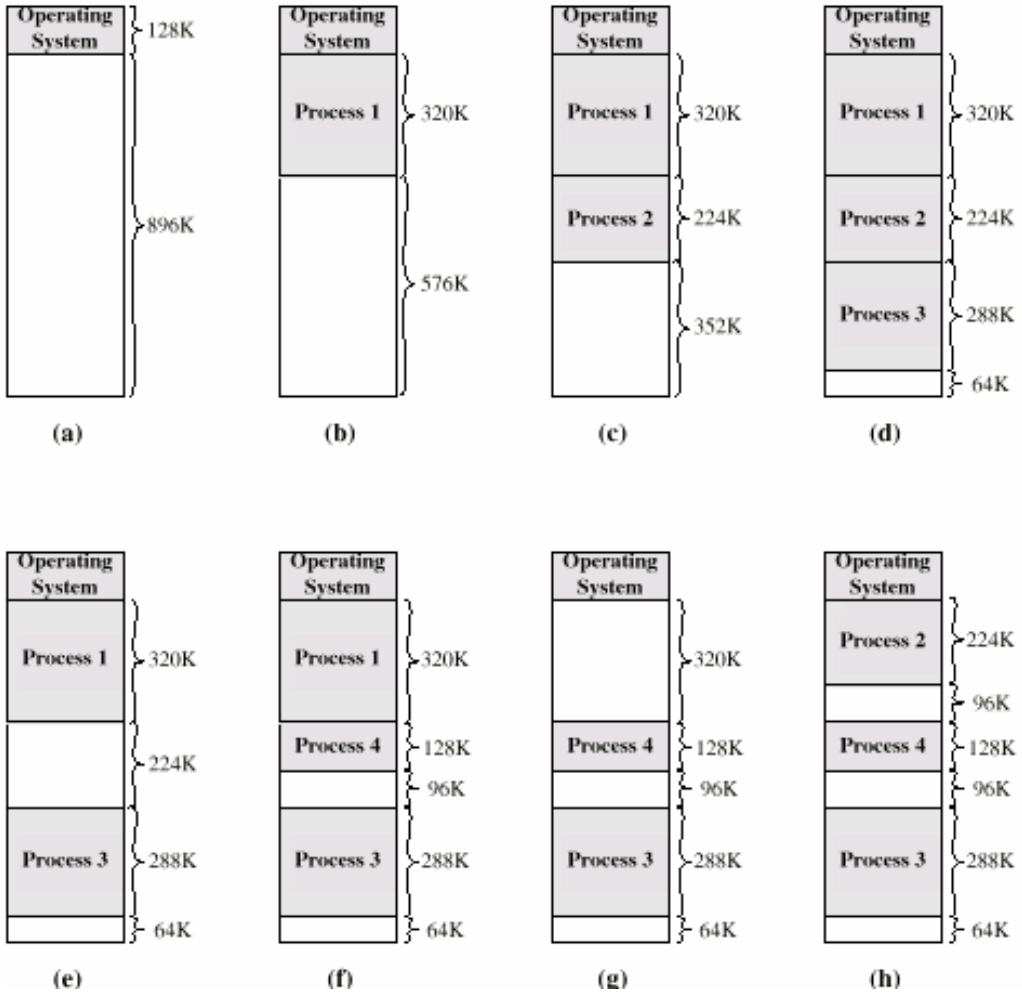


(a) Equal-size partitions (b) Unequal-size partitions

Partisi Variable-sized

- Mengalokasikan secara tepat memori yang diperlukan suatu proses.
- Menyebabkan suatu hole kosong tidak terpakai pada bagian akhir memori, yang biasanya terlalu kecil untuk digunakan.
- Dari keseluruhan memori yang tersedia, hanya ada satu hole kecil - sedikit yang tersia-siakan
- Ketika semua proses diblok, akan terjadi penukaran (swap) proses keluar CPU, lalu CPU mengambil proses yang lain.
- Proses baru mungkin saja lebih kecil dibanding proses yang ditukar keluar tadi, sehingga membuat hole baru.
- Amat cepat terbentuk sekelompok hole (fragmentation)
- Solusi:
    - Koalisi - Menggabung dengan hole yang bersebelahan ke hole yang lebih besar.
    - Pemampatan (Compaction) - Dari waktu ke waktu melakukan perpindahan semua hole ke dalam satu blok bebas (seperti pada disk de-fragmentation)

Ilustrasi Partisi Variable-sized



Relokasi

- Tidak ada jaminan suatu proses akan mengisi ke dalam tempat yang sama dengan proses yang lain di dalam memori.

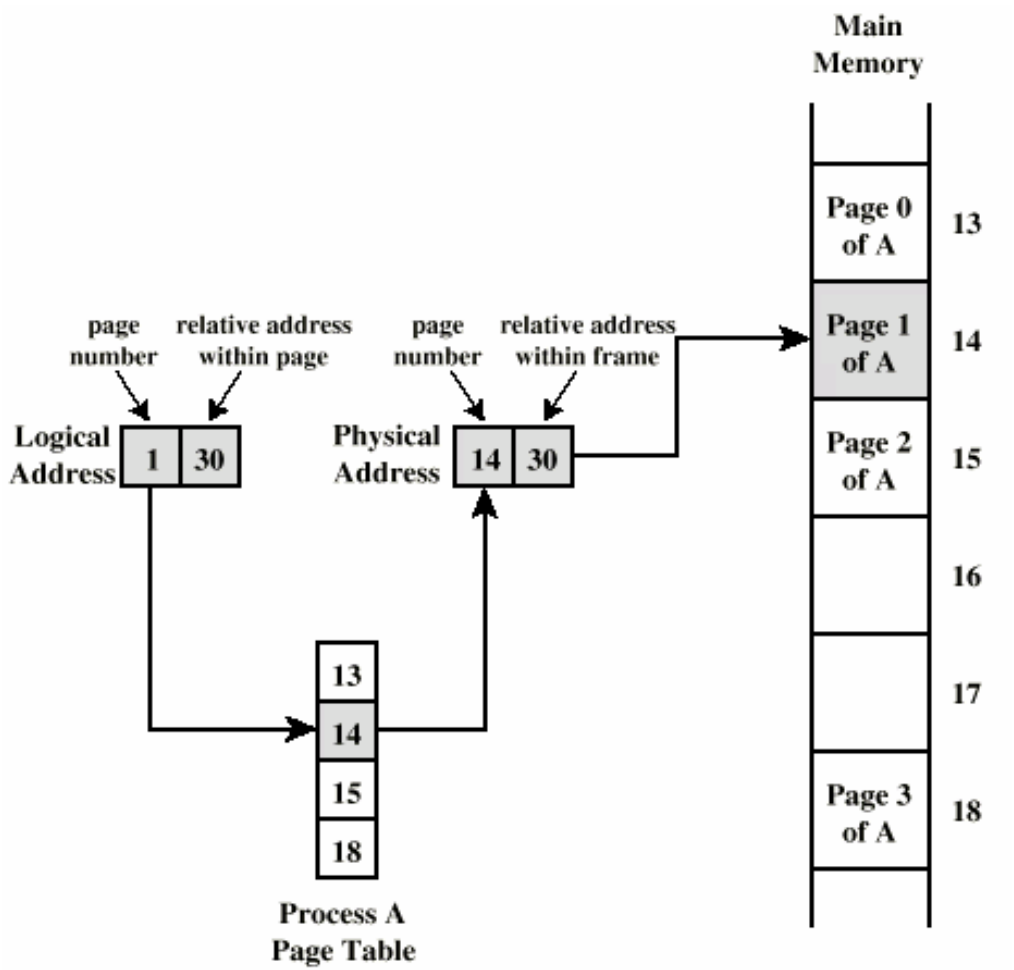
- Instruksi berisi alamat :
  - Lokasi data
  - Alamat instruksi (percabangan)

- Alamat logis - relatif untuk awal suatu program.
- Alamat fisik - Lokasi aktual (yang sesungguhnya) dalam memori pada saat ini.
- Konversi otomatis berdasarkan alamat.

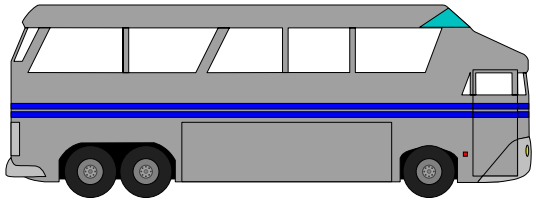
Paging

- Memori di-split pada ukuran yang sama, chunks kecil - disebut *frame Page*.
- Program yang dipisah (proses) pada ukuran yang sama, small chunks – disebut *page*.
- Mengalokasikan page frame yang diperlukan kepada suatu proses.
- Sistem operasi memelihara daftar frame yang bebas.

Ilustrasi Paging

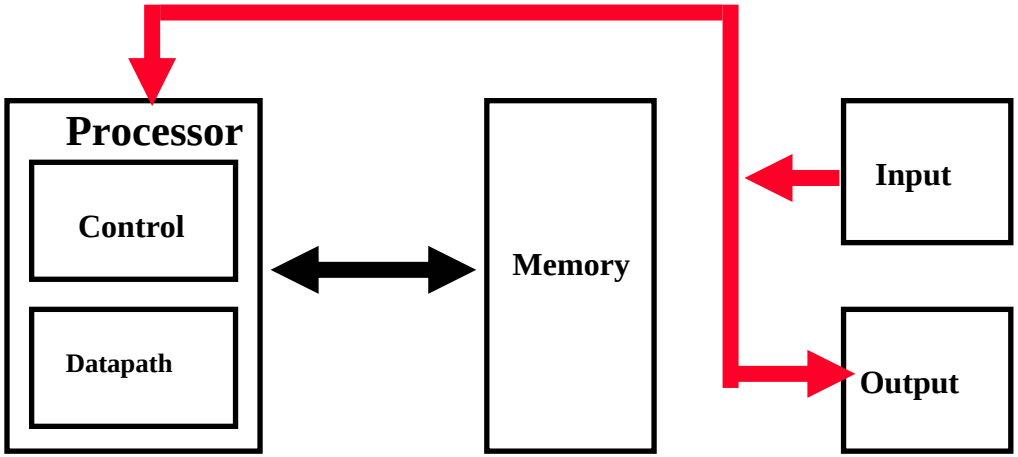


SISTEM BUS



Bus adalah :

- Jalur komunikasi yang dibagi pakai
- Suatu set kabel tunggal yang digunakan untuk menghubungkan berbagai subsistem.



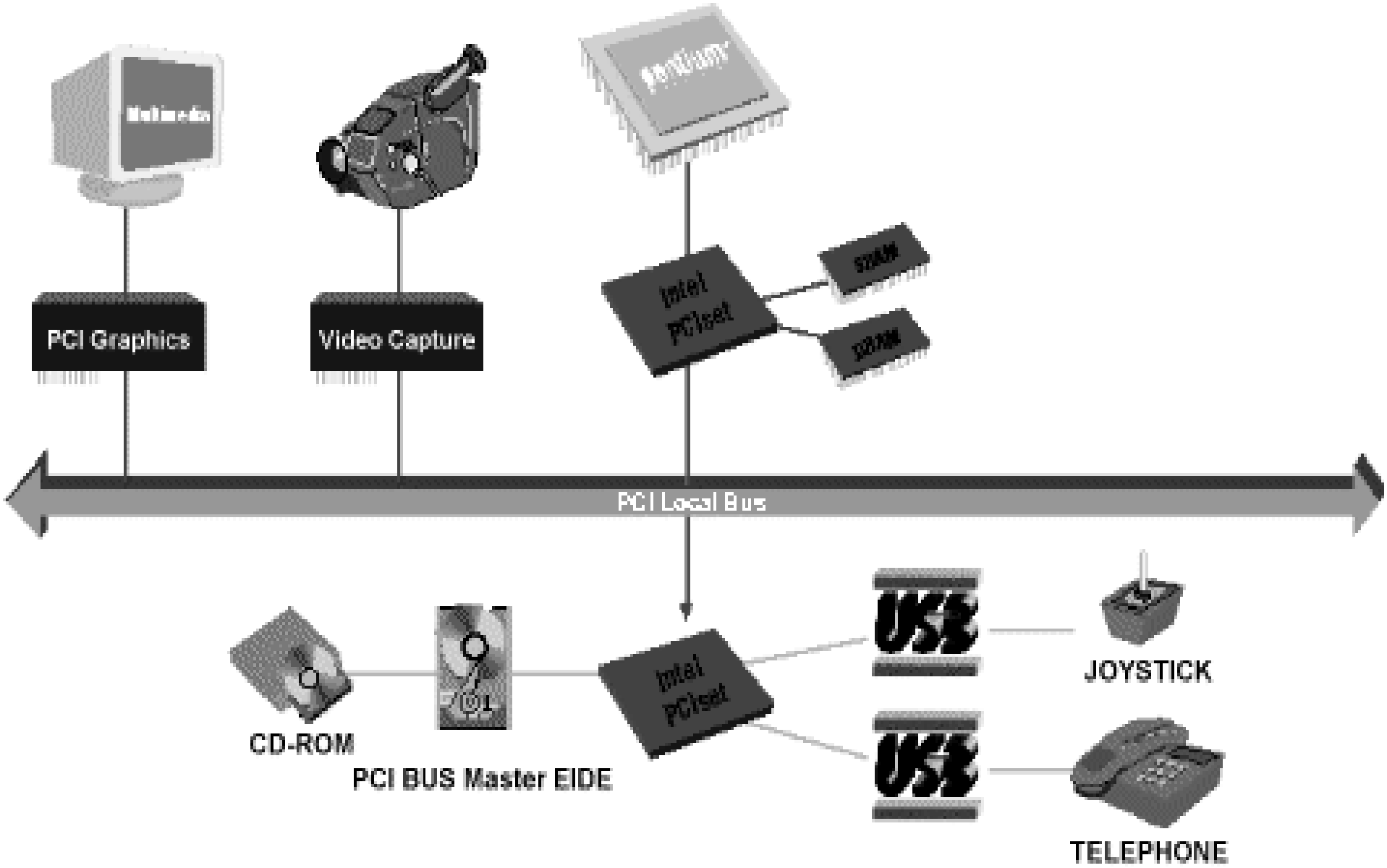
- Merupakan tool yang fundamental dalam menyusun sistem yang besar dan kompleks secara sistematis.

KEUNTUNGAN MENGGUNAKAN BUS

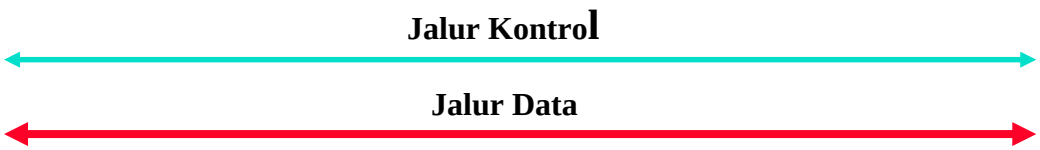
- Memiliki berbagai macam kegunaan (Versatilitas):
  - Device baru dapat ditambahkan dengan mudah.
  - Peripheral dapat dipindah-pindahkan antar sistem komputer yangh menggunakan standar bus yang sama.
- Harga Rendah:
  - Set kawat tunggal dipakai bersama dalam berbagai cara dan metode

KERUGIAN MENGGUNAKAN BUS

- Menciptakan suatu komunikasi bottleneck
- Bandwidth (Luas bidang) bus dapat membatasi maksimum I/OThroughput
- Maksimum kecepatan bus biasanya dibatasi oleh:
  - Panjang bus
  - Banyaknya device yang terhubung pada bus
- Kebutuhan dukungan tambahan untuk device-device yang :
  - Secara luas memiliki berbagai jenis latency
  - Secara luas memiliki berbagai jenis data transfer rate


PCI Graphics
Video Capture
intel
PCIset
PCI Local Bus
CD-ROM
PCI BUS Master EIDE
intel
PCIset
JOYSTICK
TELEPHONE

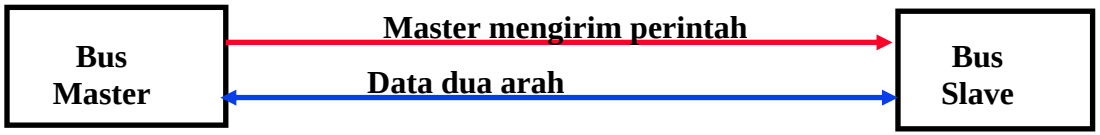
ORGANISASI BUS



- **Jalur Kontrol :**
  - **Berisi Signal request dan sinyal acknowledgments**
  - **Mengindikasikan tipe informasi pada jalur data.**

- **Jalur Data membawa informasi antara sumber dan tujuan:**
  - **Data dan alamat**
  - **Perintah-perintah kompleks.**

MASTER VS SLAVE



- Suatu **transaksi bus** meliputi dua komponen:
  - Mengeluarkan perintah (dan alamat) - request (permintaan)
  - Memindahkan data - action (tindakan)

- **Master** adalah bus yang memulai transaksi bus dengan cara :
  - Mengeluarkan perintah (dan alamat)

- **Slave** adalah bus yang bereaksi terhadap alamat dengan cara :
  - Mengirimkan data kepada master jika master meminta data.
  - Menerima data dari master jika master mengirimkan data

DIRECT MEMORY ACCESS (DMA)

- Device I/O harus memindahkan sejumlah data yang besar dari memori ke prosesor :
  - Disk harus memindahkan blok lengkap (4K? 16K?)
  - Paket besar dari jaringan
  - Daerah frame penyangga (buffer)
- DMA memberi kemampuan pada device eksternal untuk dapat menulis ke memori secara langsung: Overhead yang lebih kecil dibandingkan dengan jika prosesor me-request 1 word setiap waktu.
  - Prosesor (atau sistem memori) bertindak seperti slave

Kasus : Cache coherence
Apa Akibatnya jika device I/O menulis data yang sedang berada dalam prosesor cache? Prosesor tidak dapat lihat data baru!

Solusi:
- Membersihkan cache pada setiap Operasi I/O (mahal)
- Memiliki perangkat keras yang membuat tidak berlakunya suatu jalur cache.

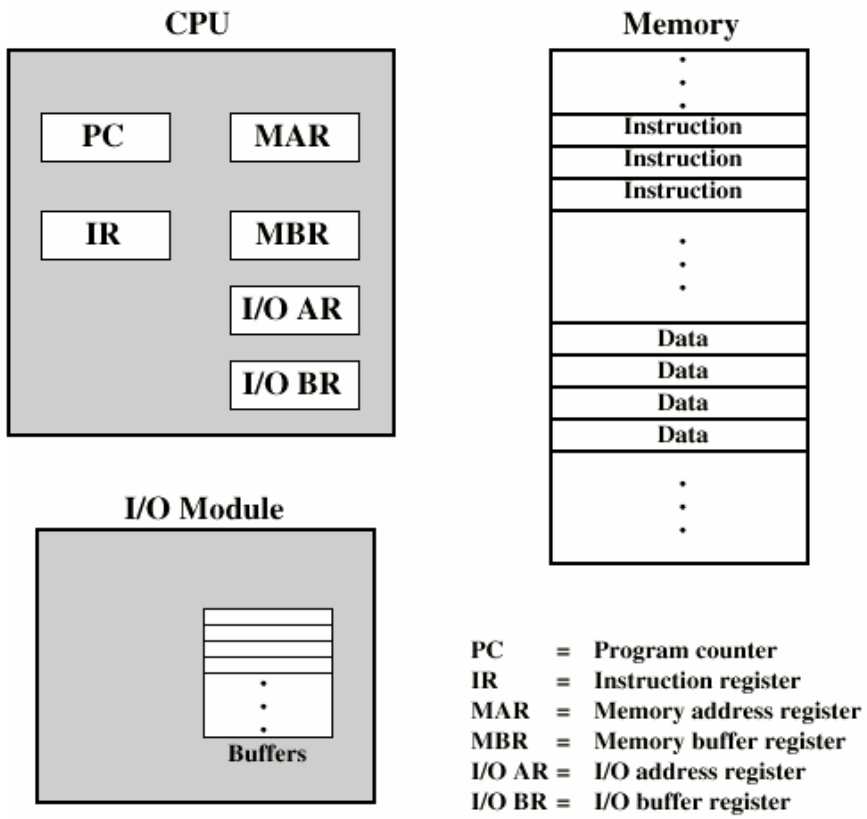
KONSEP PROGRAM

Program adalah :
- Kumpulan Langkah yang berurutan
- Untuk setiap langkah, akan menyelesaikan operasi aritmatik atau operasi logikal
- Untuk setiap operasi, membutuhkan suatu sinyal kontrol tertentu.

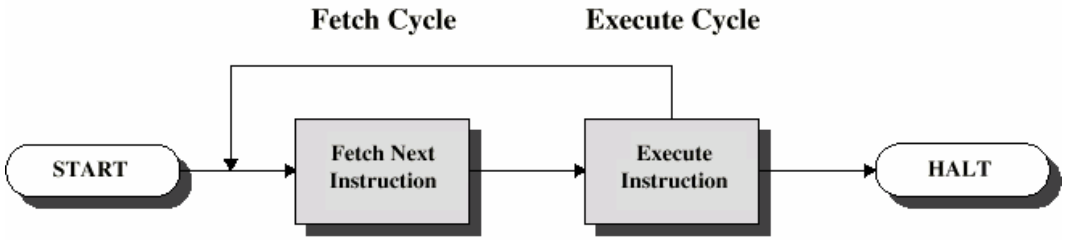
Fungsi Kontrol Unit :
- Untuk setiap operasi membutuhkan kode unik : ADD, MOVE
- Segmen hardware menerima kode tersebut dan memilihkan kontrol sinyal yang tepat.
- Kita memiliki Komputer!

Komponen :
- Control Unit dan unit Aritmatik &Logika terdapat pada Central Processing Unit
- Data dan instruksi perlu masuk ke dalam sistem tersebut dan hasilnya dikeluarkan dari sistem tersebut : Proses Input/output
- Penyimpanan sementara bagi kode dan hasil amat diperlukan, contoh : Main memory

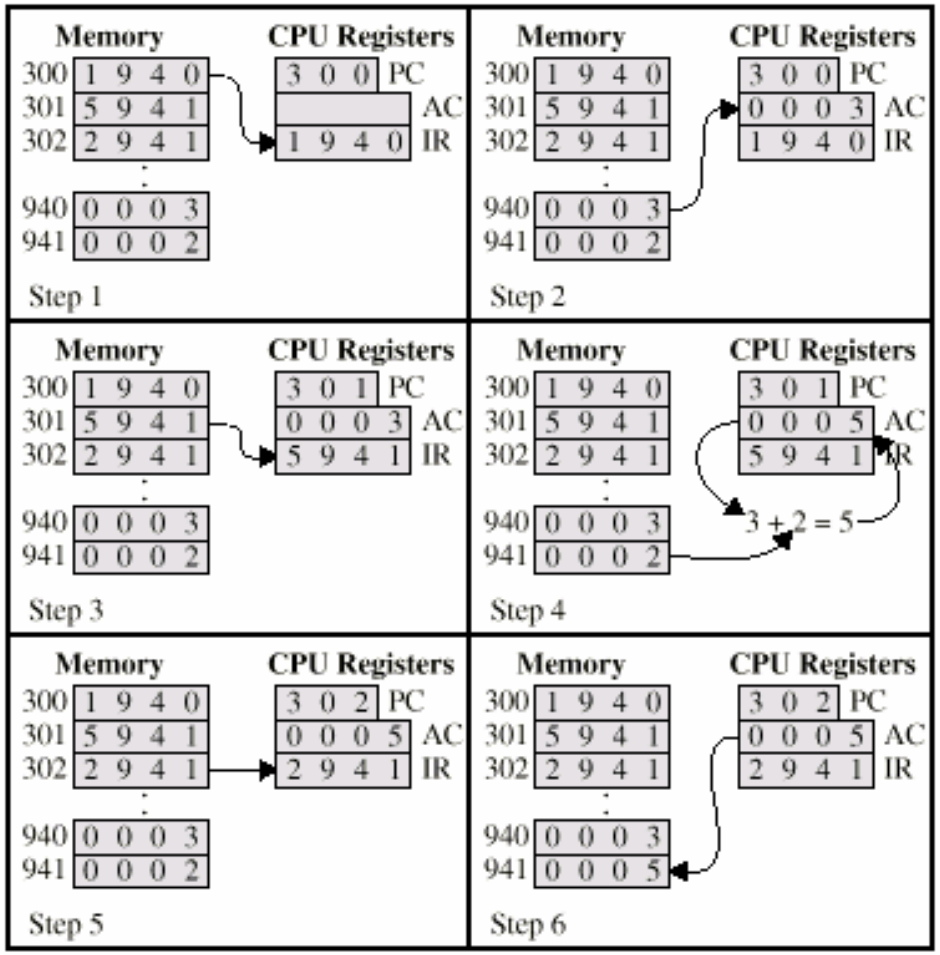
Komputer Komponen (Top Level View) :



Siklus Instruksi :



Contoh Eksekusi Program :

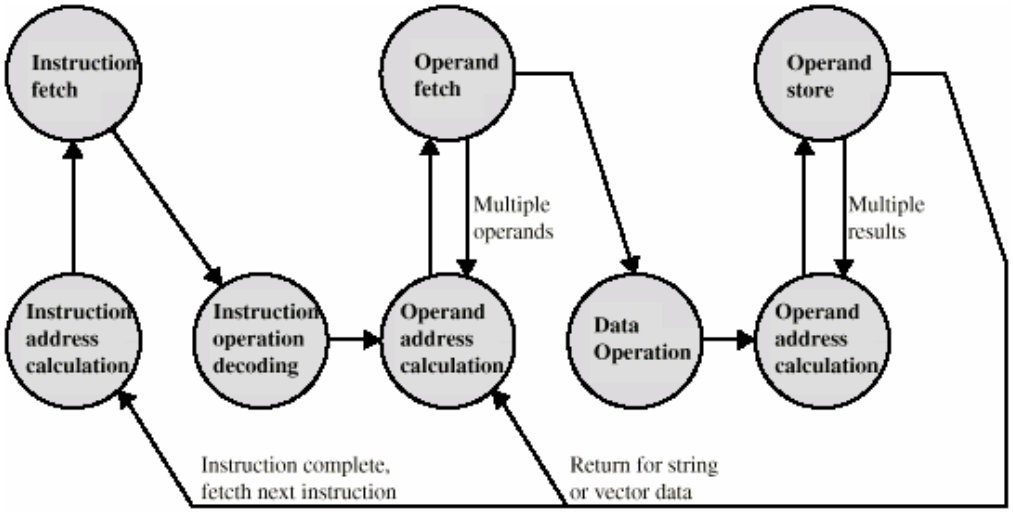


Dua Langkah Siklus Instruksi :

1. Siklus Fetch
    - Program Counter (PC) mengambil alamat instruksi selanjutnya pada fetch.
    - Processor mengambil instruksi dari memori yang ditunjuk oleh PC
    - Increment PC : mencari alamat lainnya
    - Instruksi di-load ke Instruction Register (IR)
    - Prosesor menginterpretasi instruksi dan menyiapkan aksi yang dibutuhkan

2. Siklus Eksekusi
    - Processor-memory : transfer data antara CPU dan main memory
    - Processor I/O : transfer data antara CPU dan modul I/O
    - Data processing : Beberapa operasi aritmatik dan logika pada data
    - Kendali.: alternatif aliran operasi, misalnya jump
    - Kombinasi di atas.

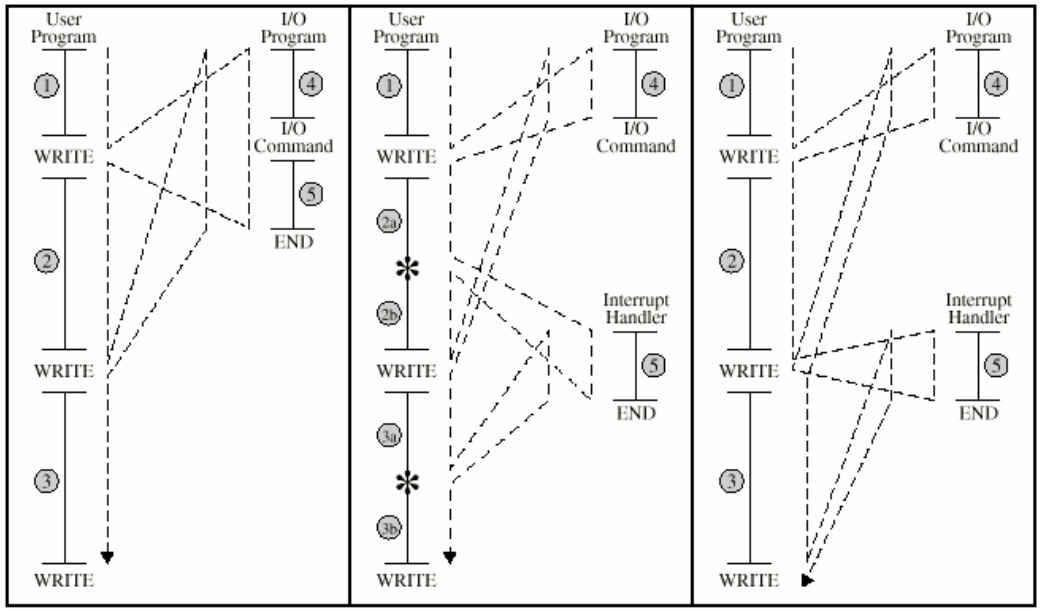
State Diagram Siklus Intruksi :



Interupsi :

- Mekanisme antara modul yang berlainan (misalnya I/O) dapat menginterupsi aliran proses yang normal.
- Berhubungan dengan program, misalnya : overflow, division by zero
- Berhubungan dengan Timer, misalnya : dibuat oleh timer pada prosesor internal, digunakan dalam multitasking
- Berhubungan dengan I/O : dari I/O controller
- Berhubungan dengan kesalahan Hardware, misalnya error pada parity memori.

Kendali Aliran Program :

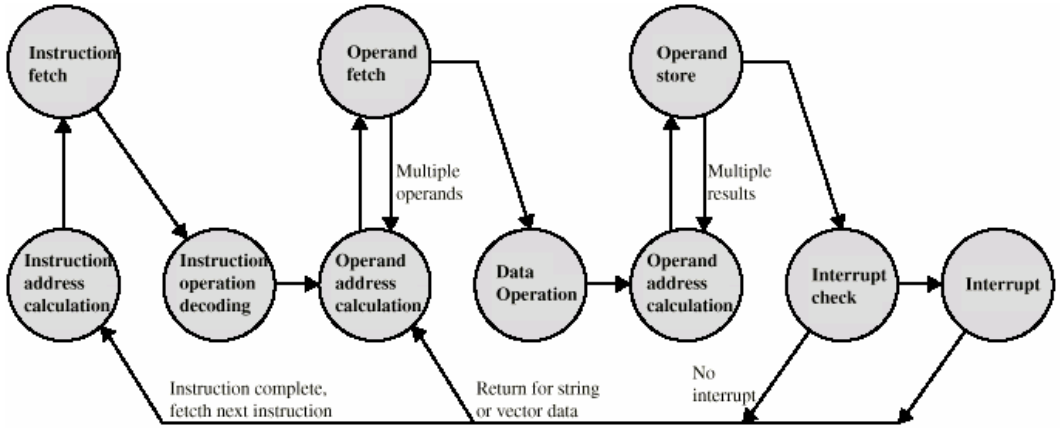


(a) No interrupts (b) Interrupts; short I/O wait (c) Interrupts; long I/O wait

Siklus Interupsi :

- Ditambahkan dalam siklus Instruksi
- Prosesor akan menmeriksa interupsi : diindikasikan dengan adanya sinyal interupsi.
- Jika tidak ada interupsi maka akan mem-fetch intruksi selanjutnya.
- Jika terdapat interupsi :
  - Hentikan dulu eksekusi program yang sedang berjalan
  - Isi program di simpan dulu
  - Set PC untuk menunjuk ke alama awal rutin handler interupsi
  - Memproses interupsi.
  - Mengembalikan isi program dan melanjutkan program yang terinterupsi tadi.

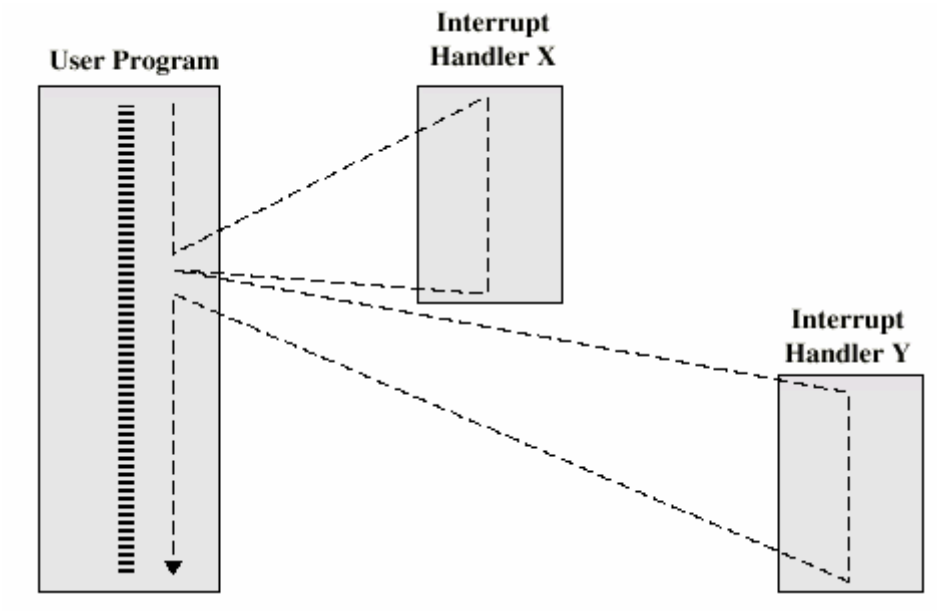
State Diagram Siklus Interupsi :



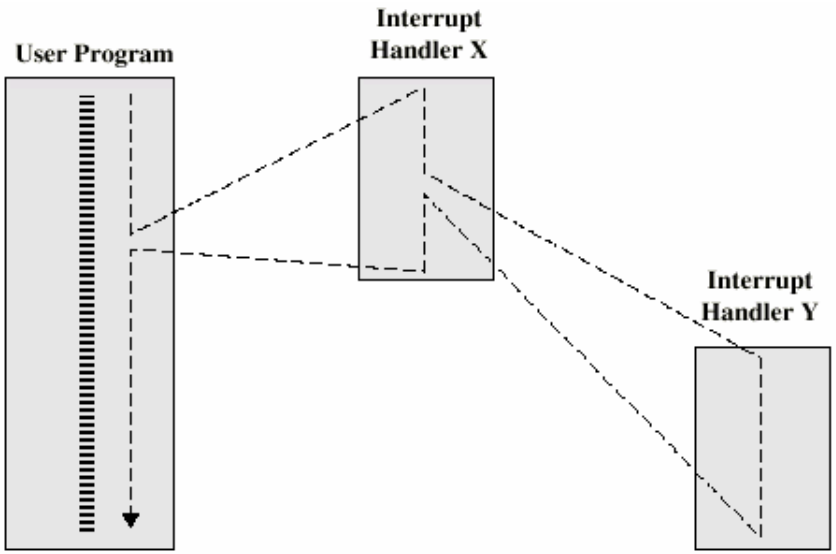
Multiple Interupsi :

- Men-disable interupsi
  - Prosesor akan mengabaikan interupsi yang lain kerika sedang memproses suatu interupsi
  - Interupsi yang di-pending akan dijalankan setelah interupsi yang sebelumnya telah selesai diproses.
  - Terdapat Interupsi yang menangani urutan.
- Mendefinisikan prioritas
  - Interupsi prioritas rendah dapat diinterupsi oleh interupsi yang lebih tinggi prioritasnya.
  - Ketika proses pada interupsi yang berprioritas tinggi selesai dilakukan, proses akan kembali ke interupsi sebelumnya.

Model Sequensial Multiple Interupsi :



Model Nested Multiple Interupsi :



KONEKSI

- Setiap unit harus terkoneksi
- Tipe koneksi berbeda untuk setiap tipe unit yang dikoneksikan.

Koneksi Memori :

- Receive dan send data
- Receive alamat (suatu lokasi)
- Receive sinyal kontrol : Read, Write, Timing

Koneksi Input/Output :

- Hampir mirip dengan koneksi memori
- Output : Receive data dari computer dan Send data ke peripheral.
- Input : Receive data dari peripheral dan Send data ke computer
- Receive sinyal kontrol dari komputer
- Send sinyal kontrol ke peripheral, contoh : ke disk.
- Receive alamat dari komputer, contoh : nomor port untuk identifikasi peripheral.
- Send sinyal interupsi (kontrol)

Koneksi CPU :

- Read instruksi dan data.
- Write out data (setelah diproses).
- Send sinyal kontrol ke unit lain.
- Receive (dan menjalankan) interupsi

MACAM KONEKSI BUS

BUS DATA :

- Membawa data : Dalam hal ini tidak ada perbedaan antara data dan instruksi.
- Besar jalur ini mempengaruhi kinerjanya, contoh : 8, 16, 32, 64 bit.

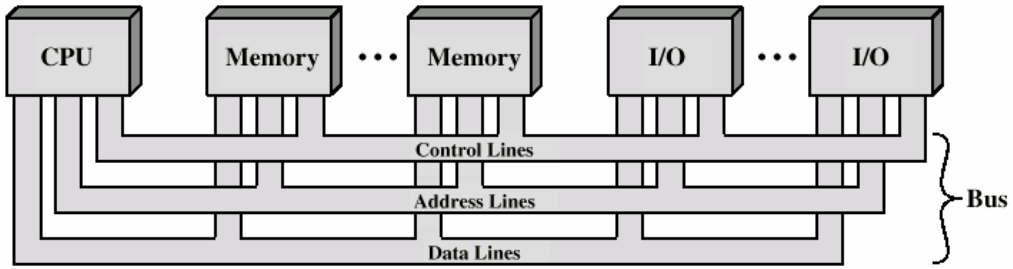
BUS ALAMAT :

- Identifikasi sumber atau tujuan data
- Contoh : CPU perlu membaca instruksi (data) dari lokasi yang diberikan dalam memori.
- Lebar Bus menunjukkan kapasitas maksimum memori suatu sistem, contoh : 8080 memiliki 16 bit bus alamat yang memberikan 64k space alamat

BUS KONTROL :

- Mengendalikan dan mengatur timing informasi :
  1. Sinyal read/write memori.
  2. Interrupt request
  3. Sinyal Clock

SKEMA KONEKSI BUS :

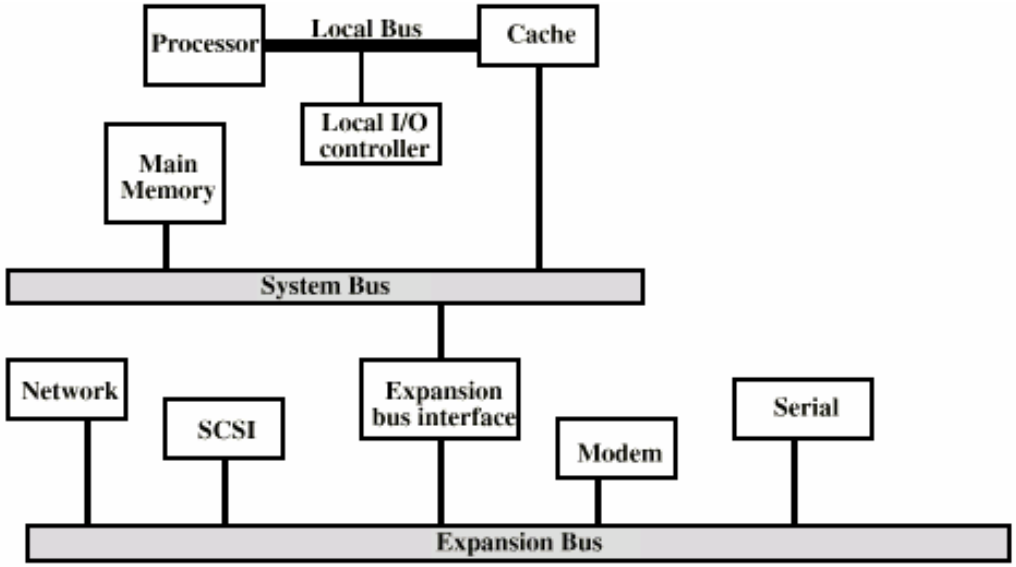


KELEMAHAN SINGLE BUS :

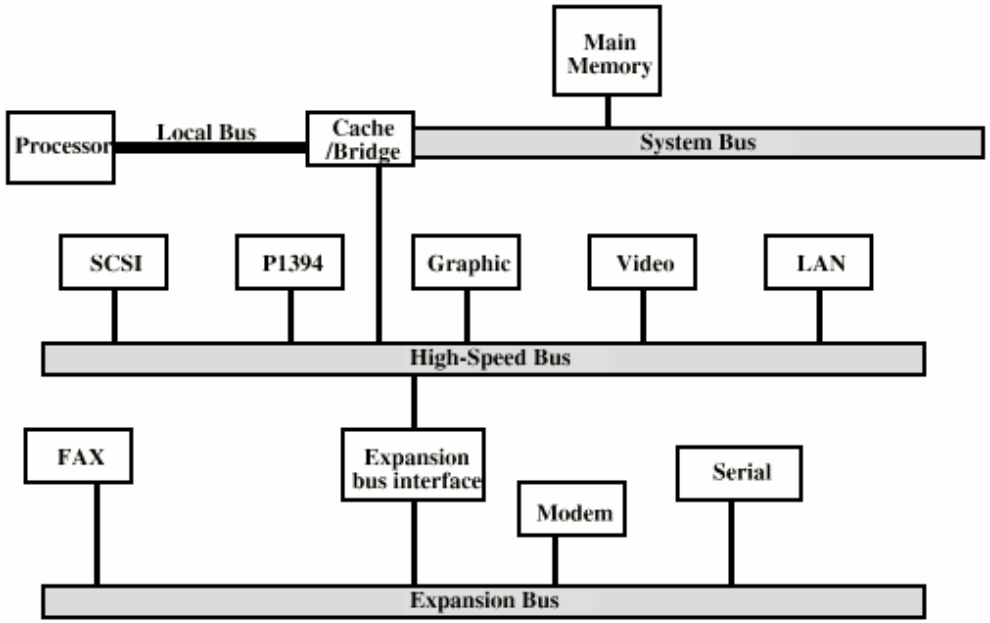
Banyak device dalam 1 bus akan membuat :

- Delay Propagasi :
  - Data path yang panjang mengakibatkan koordinasi penggunaan bus mempengaruhi kinerjanya.
  - Jika data transfer agregat, akan memenuhi kapasitas bus dengan cepat.

BUS TRADISIONAL ISA (DENGAN CACHE) :



BUS DENGAN KINERJA TINGGI :



TIPE BUS

Menurut Berkeley :

- **Processor-Memory Bus (desain spesifik)**
  - pendek dan berkecepatan tinggi
  - Hanya dibutuhkan untuk berpasangan dengan sistem memori.
  - Memaksimalkan bandwidth memori-ke-prosesor.
  - Menghubungkan secara langsung dengan prosesor.
  - Mengoptimalkan transfer blok cache.

- **I/O Bus (standard industri)**
  - Pada umumnya adalah lebih lambat dan panjang
  - Diperlukan untuk berpasangan dengan berbagai macam device I/O.
  - Menghubungkan antara processor-bus memori atau dengan bus backplane.

- **Backplane Bus (kepemilikan atau standard)**
  - Backplane: suatu struktur interkoneksi di dalam chasis
  - Mengijinkan prosesor, memori, dan device I/O untuk dapat eksis pada waktu bersamaan
  - Keuntungan harga: satu bus untuk semua komponen

Menurut Stalling :

- Dedicated : Separasi jalur data dan alamat
- Multiplexed
  - Men-Share jalur
  - Kontrol jalur alamat valid atau data valid.
  - Kelebihan : jalur lebih banyak.
  - Kekurangan : Perlu kontrol lebih kompleks.

TIMING BUS

- Untuk Koordinasi setiap even dalam bus.
- Synchronous :
  - Even ditunjukkan oleh sinyal clock.
  - Bus Kontrol termasuk dalam clock line
  - Semua device dapat membaca clock line
  - Biasanya 1 siklus untuk setiap even.

Diagram Timing Synchronous :

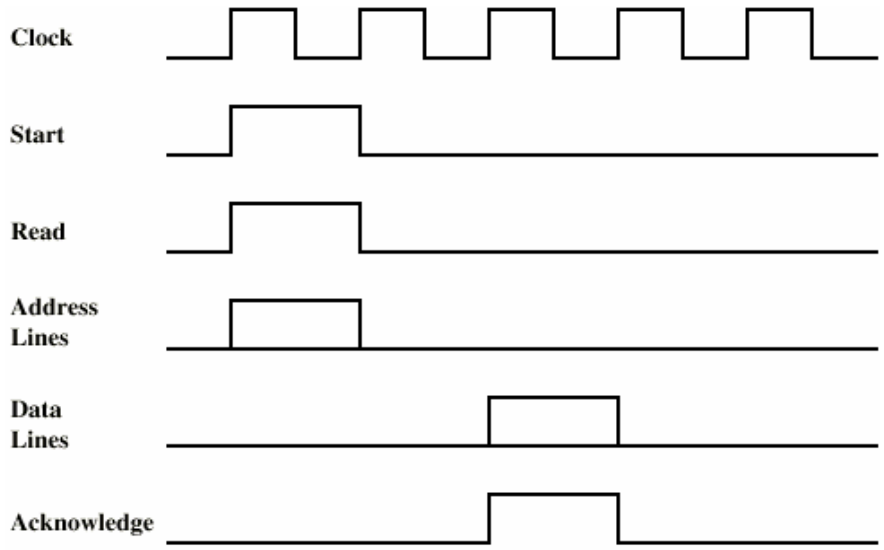
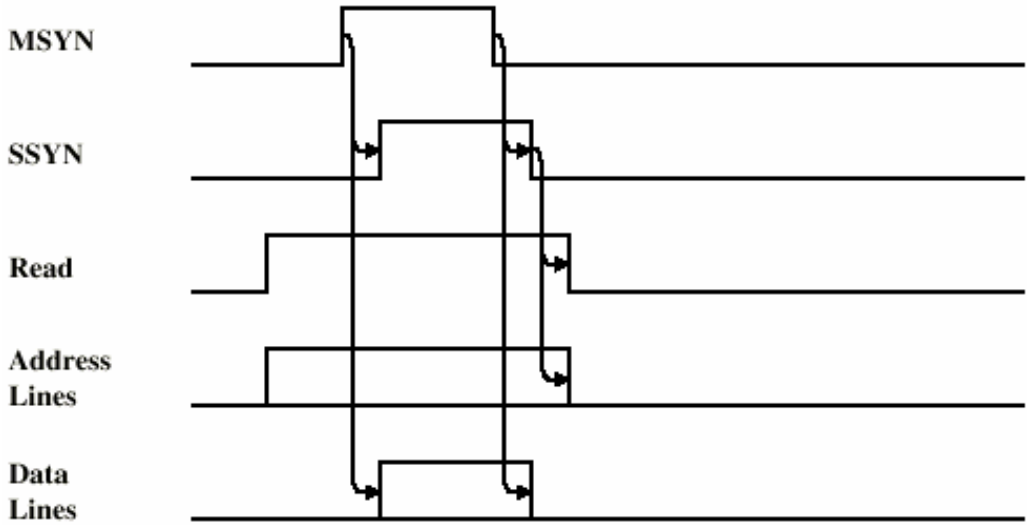
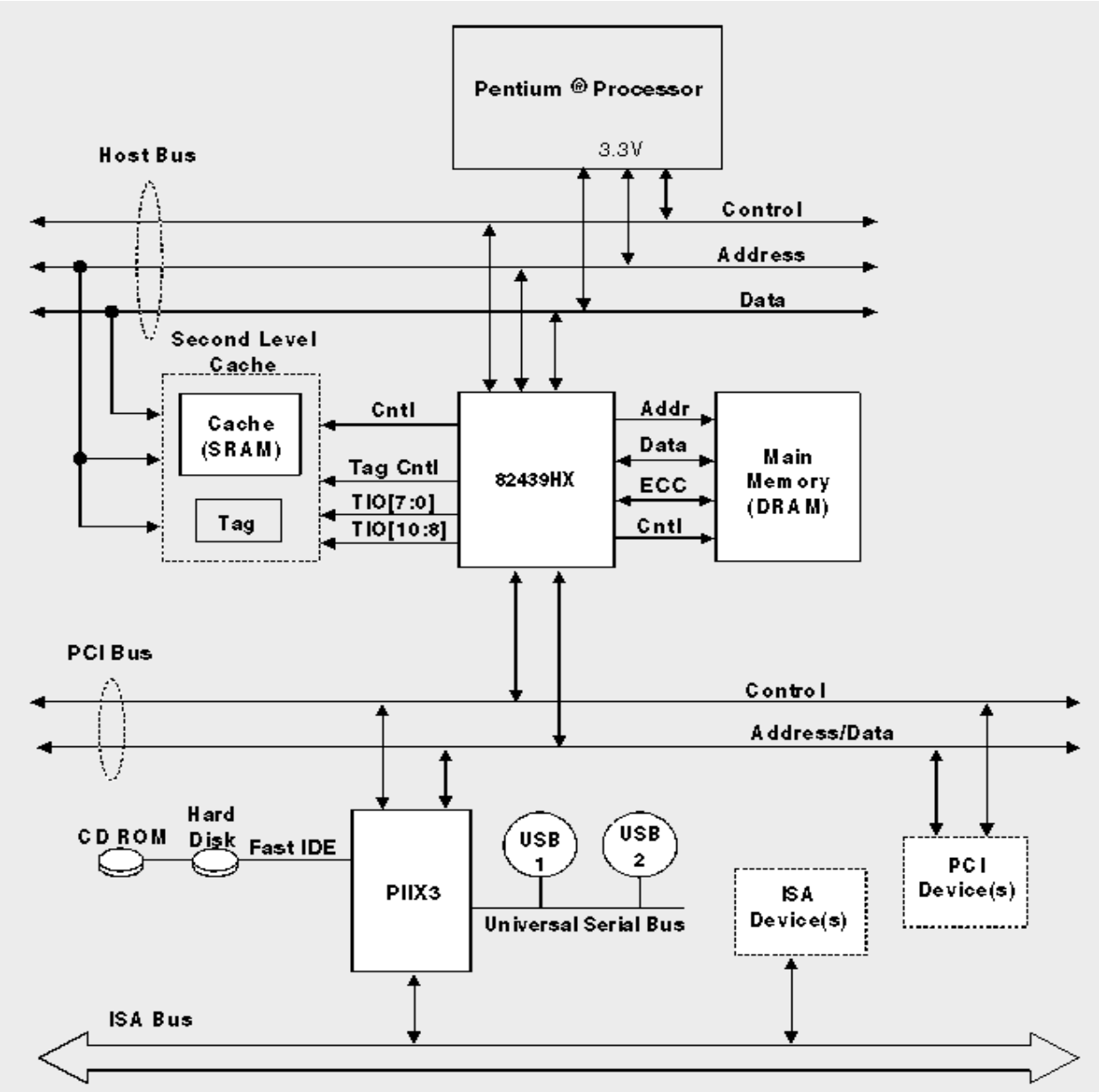


Diagram Timing Asynchronous :



Contoh : Organisasi Bus Sistem Pentium



Bus Prosesor/Memori

Bus PCI

Berbagai macam bus I/O

CPU (CENTRAL PROCESSING UNIT)

Fungsi Utama CPU :

- Fetch Instruksi
- Interpret Instruksi
- Fetch data
- Process data
- Write data

Register :

- CPU harus memiliki space untuk pengolahan yang dilakukannya (penyimpanan sementara) yang disebut register
- Jumlah dan fungsi penyimpanannya sangat bervariasi tergantung pada desain prosesornya
- Merupakan Top level dari hirarki memori.

Beberapa Fungsi Register :

1. General Purpose

- Untuk berbagai keperluan dasar
- Dapat digunakan untuk data atau pengalamatan pula, contoh :
  - untuk data : Accumulator
  - Untuk alamat : Segment

- Dengan membuat register berfungsi sebagai general purpose akan :
  - Meningkatkan fleksibilitas dan pilihan bagi programmer
  - Meningkatkan ukuran instruksi dan kompleksitas
- Dengan membuat register berfungsi secara khusus, akan membuat
  - Instruksi lebih kecil sehingga cepat.
  - Kurang fleksibel

Seberapa besar Register itu ?

- Cukup besar untuk menangani alamat.
- Cukup besar untuk menangani word
- Harus memungkinkan untuk mengkombinasikan antara dua register data seperti :Single, double, extended

2. Register Data

3. Register Alamat

4. Register Kode Status Kondisi

- Set dari bit individual : hasil dari operasi terakhir sebelumnya adalah nol.
- dapat dibaca oleh program, contoh : Jump if zero
- Biasanya tidak dapat di set oleh program.

Register Kontrol dan Status

- Program Counter
- Instruction Decoding Register
- Memory Address Register
- Memory Buffer Register

Beberapa Contoh Word Status Program :

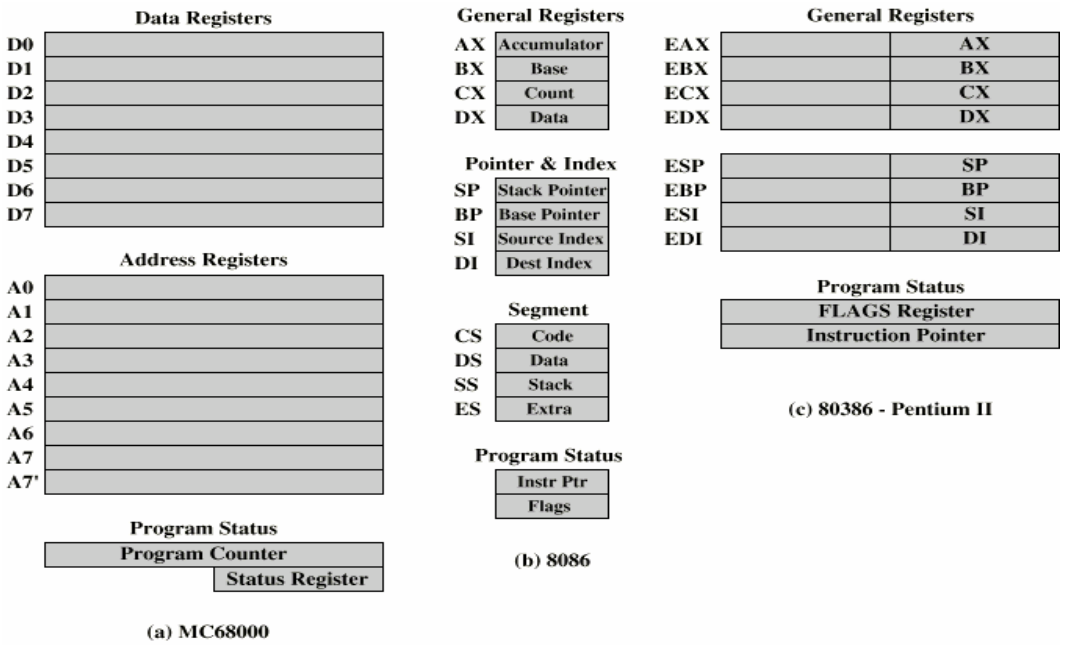
- set dari bit
- Kode kondisi
- Tanda dari hasil sebelumnya
- Zero

- Carry
- Equal
- Overflow
- Interrupt enable/disable
- Supervisor

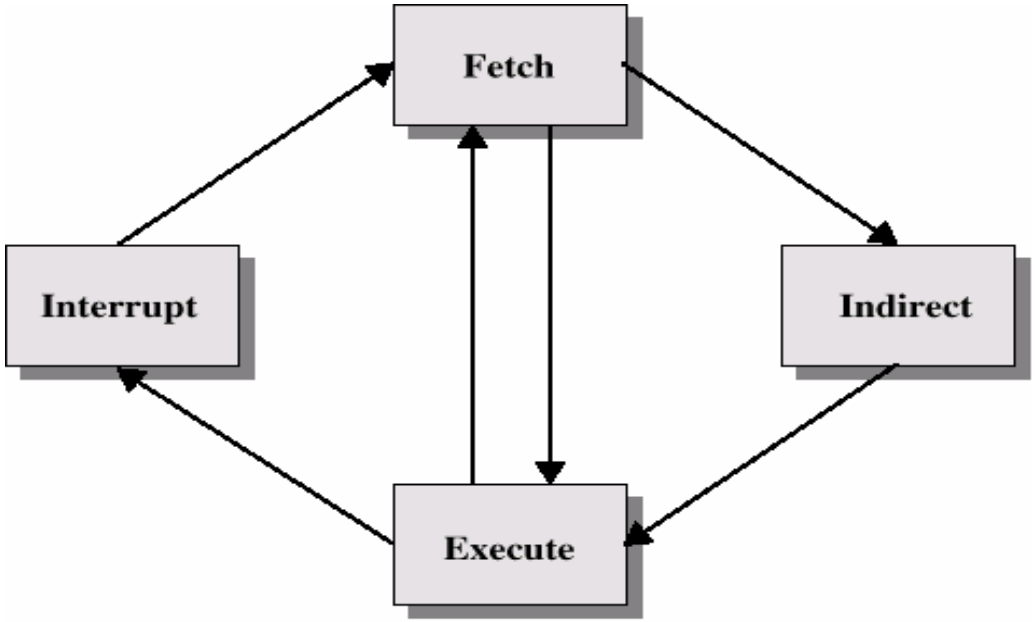
Mode Supervisor :

- Bagian yang merupakan dasar dari informasi yang diberikan oleh vendor.
- Mode Kernel
- Memberikan izin/hak (privileged) untuk mengeksekusi suatu instruksi
- Digunakan oleh sistem operasi
- Tidak tersedia bagi program user dimodifikasi.

Contoh Organisasi Suatu Register :

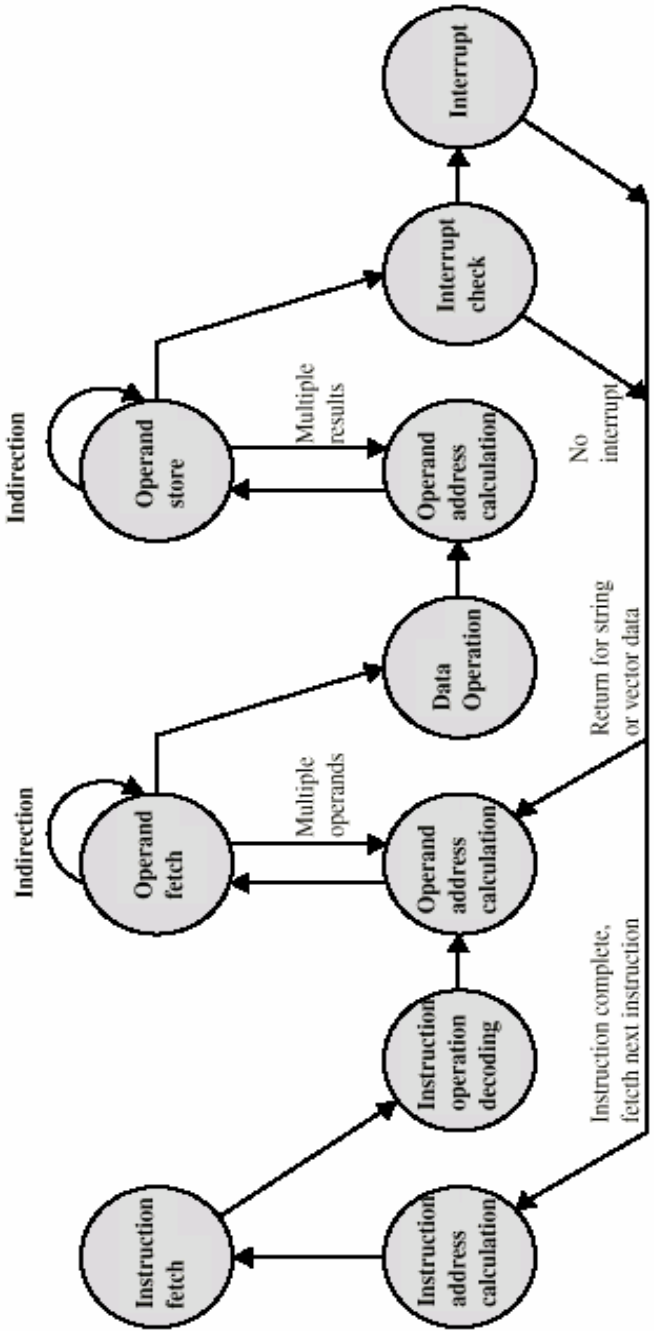


Siklus Instruksi Indirect



- Membutuhkan kses memori untuk melakukan fetch operand
- Pengalamatan Indirect membutuhkan lebih banyak akses memori
- Dapat digunakan sebagao tambahan instruksi dalam subsiklus.

Diagram Siklus Instruksi



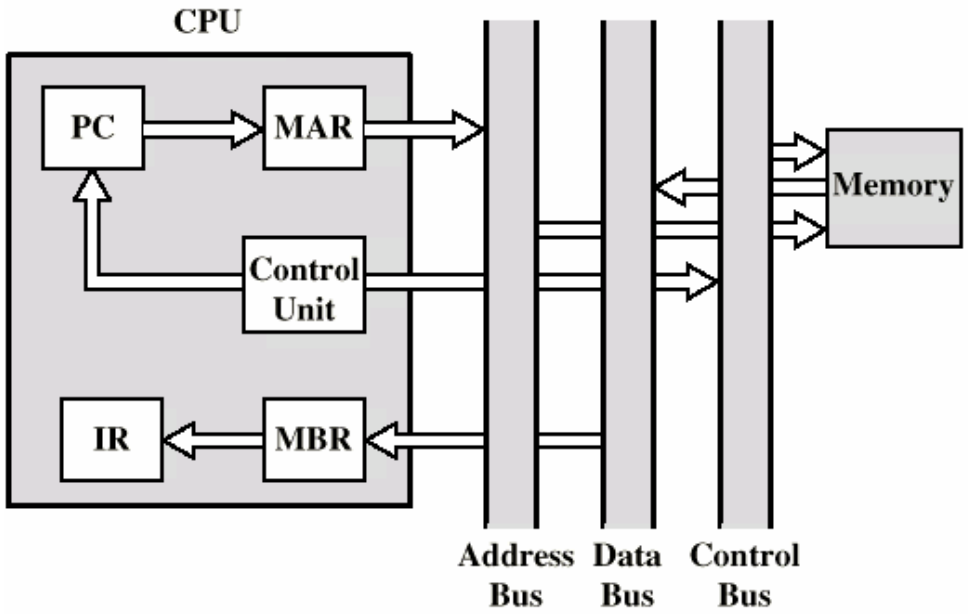
Aliran Data dalam Proses Fetch Instruksi

- Tergantung desain CPU
- Fetch
  - PC mengandung alamat instruksi selanjutnya
  - Alamat akan dipindahkan ke MAR
  - Alamat diletakkan pada bus alamat
  - Control unit meminta pembacaan memori
  - Hasil akan diletakkan pada bus dara, dicopy ke MBR kemudian ke IR
  - Biasanya PC kemudian bertambah 1

Aliran Data dalam Proses Fetch Data

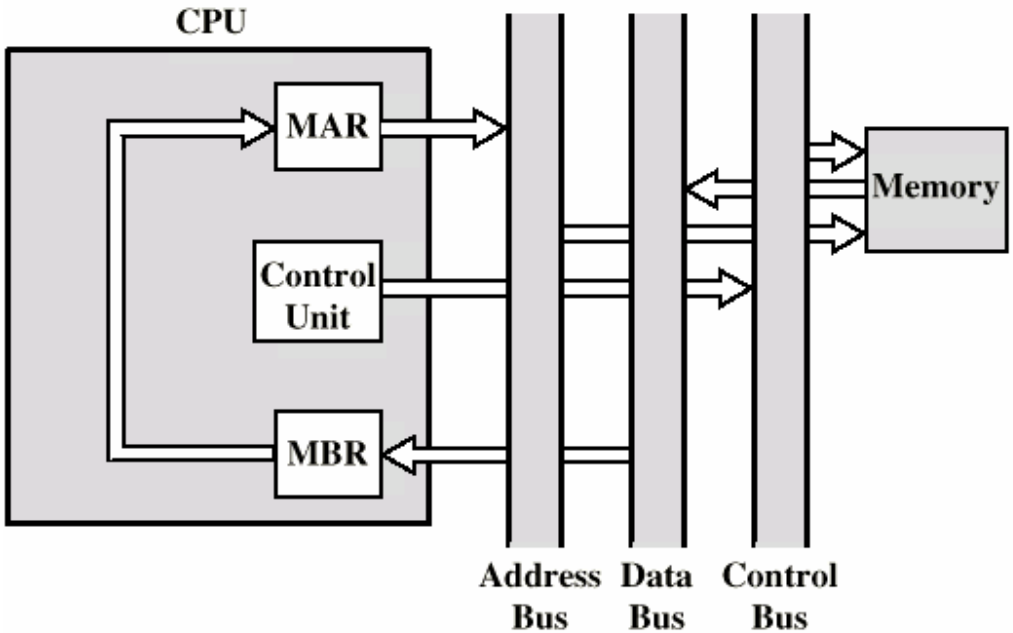
- Register IR akan diuji
- Jika pengalamatan indirect, akan mengalami siklus indirect :
  - N bit pada MBR ditransfer ke MAR
  - Control unit meminta pembacaan memori
  - Hasil (alamat operand) dipindahkan ke MBR

Diagram Aliran data Pada Proses Fetch



MBR = Memory buffer register
MAR = Memory address register
IR = Instruction register
PC = Program counter

Diagram Aliran data Pada Proses Indirect



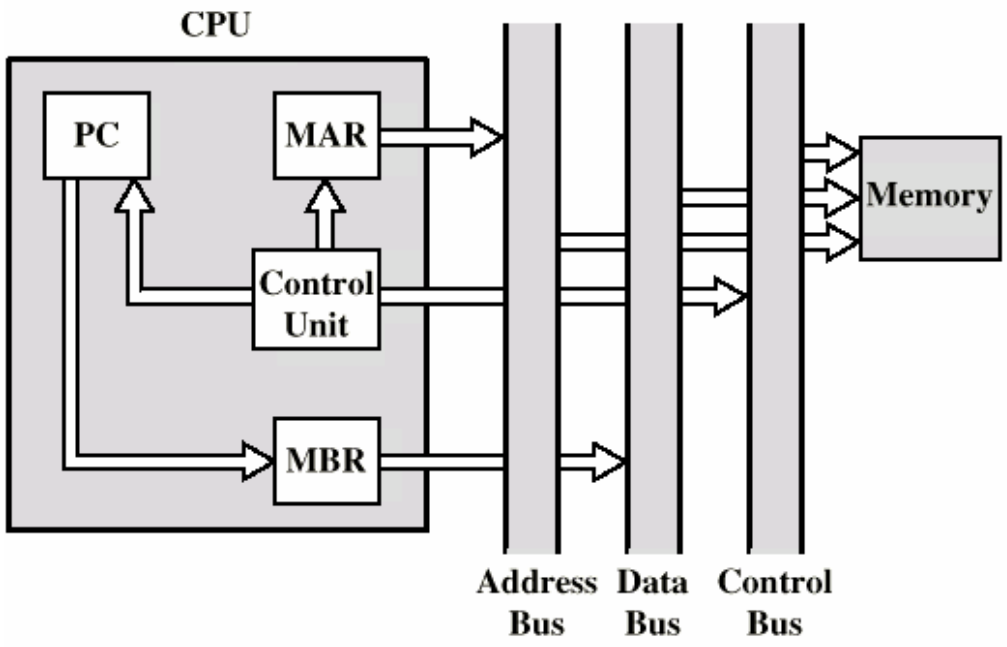
Aliran Data dalam Siklus Eksekusi

- Akan mengambil banyak form
- Tergantung pada intruksi yang sedang dieksekusi
- Biasanya didalamnya terdapat :
    - Memory read/write
    - Input/Output
    - Register transfers
    - ALU operations

Aliran Data dalam Interupsi

- Sederhana
- Dapat diprediksi
- Isi register PC akan disimpan sementara untuk kemudian dilanjutkan setelah interupsi selesai dilayani.
- Isi register PC di-copy ke MBR
- Lokasi memori spesial (misal : stack pointer) akan di-load ke MAR
- MBR akan melakukan penulisan ke memori
- Register PC di-load dengan alamat rutin yang menangani interupsi
- Instruksi selankjutnya (bagian awal dari rutin interrupt handler) dapat mulai di-fetch.

Diagram Aliran data dalam Interupsi



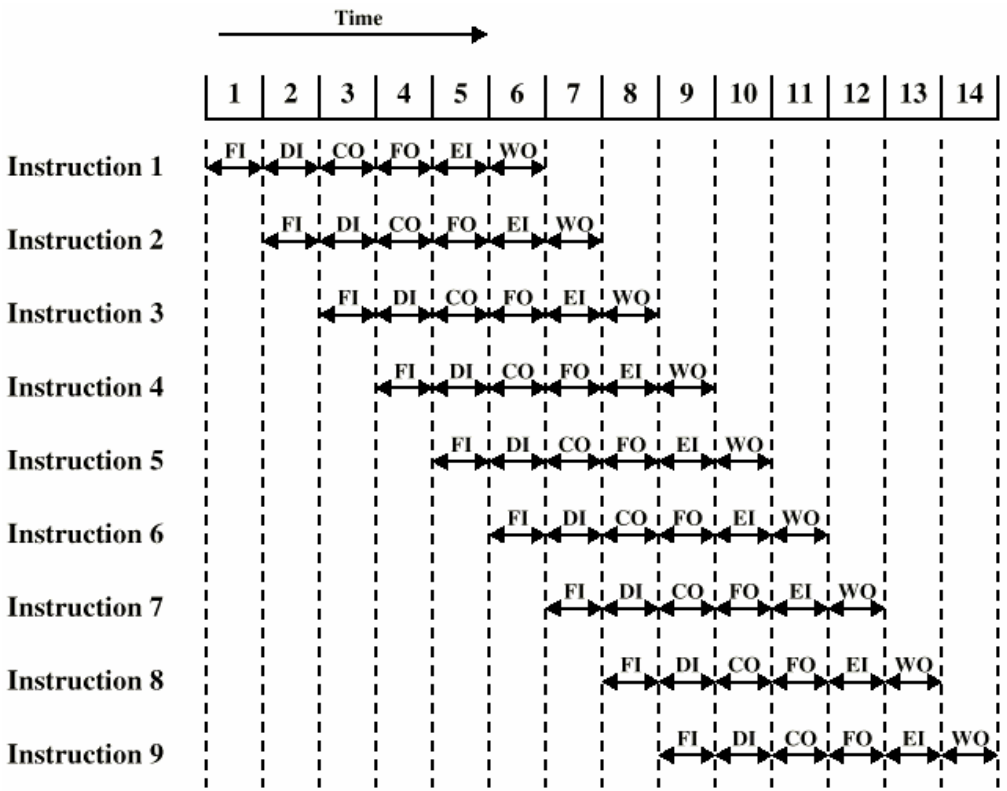
Pre-Fetch

- Siklus Fetch mengakses memori utama
- Siklus eksekusi biasanya tidak lagi mengakses memori utama
- Pada saat itulah dapat dilakukan proses fetch instruksi selanjutnya pada saat siklus eksekusi sedang berlangsung mengerjakan suatu intruksi. Inilah yang disebut instruksi prefetch.
- Fetch biasanya lebih singkat waktunya dibandingkan eksekusi, maka memungkinkan terjadinya Prefetch, namun Setiap menemui proses lompatan atau percabangan berarti proses instruksi pre-fetch yang melompat secara otomatis dalam urutan program utama seringkali tidak dibutuhkan lagi. Karena saat itu pointer prosesor sudah mengarah ke alamat percabangan yang lain lagi

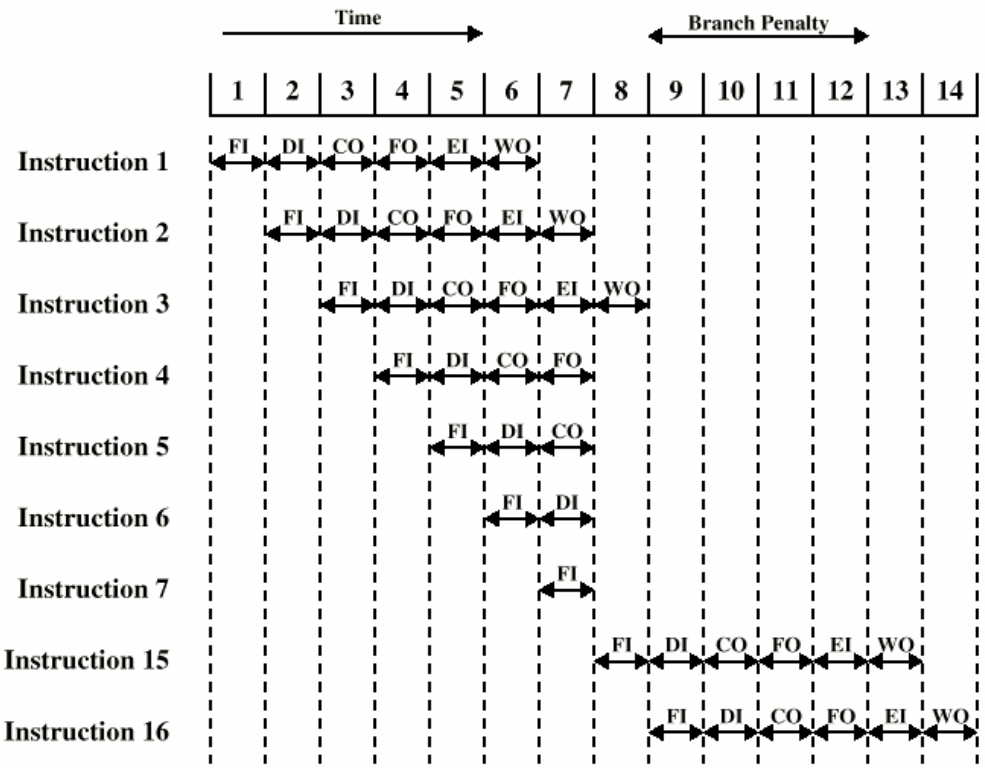
Proses Pipelining

1. mem-fetch instruksi
2. men-decode instruksi
3. Mengkalkulasi operand
4. mem-Fetch operand
5. meng-eksekusi instruksi
6. Menulis hasil
7. Melakuan overlap terhadap operasi ini dengan operasi lainnya.

Timing Pada Proses Pipelining



Percabangan Pada Pipelining



Penanganan Percabangan

1. Multiple Streams
2. Prefetch Target percabangan
3. Loop buffer
4. Memprediksi percabangan
5. Delay percabangan

1. Multiple Stream

- Memiliki dua pipeline
- Prefetch setiap percabangan akan masuk ke pipeline tersebut
- Kemudian disalurkan ke busmelalui register
- Percabangan yang banyak akan diarahkan ke pipeline secara bergantian

2. Pre-fetch Target Percabangan

- Tujuan percabangan di prefetch dengan instruksi tambahan yang diberikan pada saat ditemukan percabangan.
- Infromasi target akan dipertahankan sampai terjadi eksekusi di percabangan.
- Contoh penggunaannya pada mesin IBM 360/91

3. Loop buffer

- Memanfaatkan memori yang sangat cepat sebagai buffer
- Ditangani oleh proses fetch pada pipeline
- Akan selalui melihat dulu isi buffer sebelum melakukan fetch ke memori
- Sangat baik untuk looping yang kecil atau lompatan
- Contoh penggunaannya pada mesin CRAY-1

4. Memprediksi percabangan

Prediksi yang tidak akan terlaksana (Predict never taken)

Mengasumsikan bahwa suatu lompatan tidak akan terjadi

- Selalu mem- fetch instruksi selanjutnya
- Misalnya : mesin VAX tidak akan melakukan prefetch setelah percabangan jika terjadi kesalahan page pada hasilnya

Prediksi yang akan terlaksana (Predict always taken)

- Mengasumsikan bahwa suatu lompatan akan terjadi
- Selalu mem-fetch instruksi untuk target

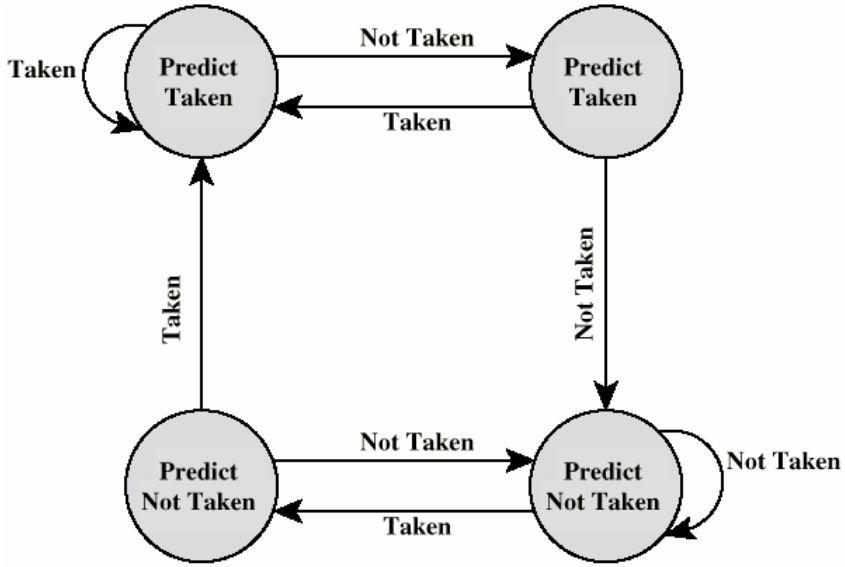
Prediksi berdasarkan Opcode

- Sejumlah instruksi menjadikan suatu lompatan menjadi lebih mungkin dilaksanakan
- Dapat mencapai kesuksesan 75%

Switch Taken/Not taken

- Berdasarkan pada histry sebelumnya
- Sangat baik untuk looping

State Diagram Branch Prediction



5. Delay Percabangan

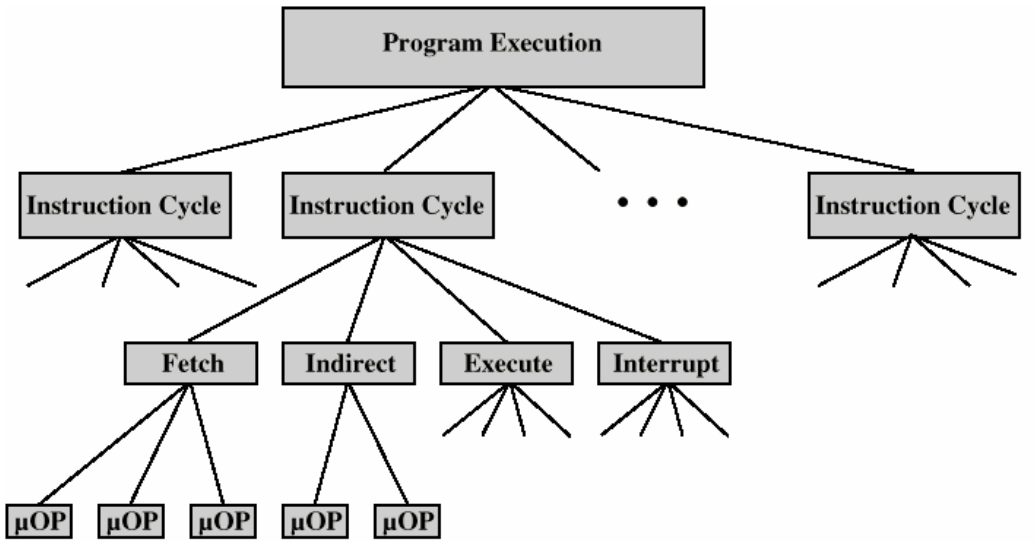
Tidak akan melompat sampai selesai menyusun kembali intruski dengan sempurna.

CU (CONTROL UNIT)

Operasi Mikro

- Komputer mengeksekusi program
- Mengalami Siklus Fetch/execute
- Setiap siklus memiliki beberapa langkah, inilah yang disebut operasi mikro
- Operasi Atomik CPU

Elemen dalam Eksekusi Program



Proses Fetch - 4 Register

- Memory Address Register (MAR)
  - Terkoneksi dengas bus alamat
  - Menetapkan alamat untuk operasi read atau write
- Memory Buffer Register (MBR)
  - Terkoneksi dengas bus data
  - Mempertahankan data untuk ditulis atau dibaca sampai selesai.
- Program Counter (PC)
  - Mempertahankan alamat instruksi selanjutnya yang akan di fetch kemudian
- Instruction Register (IR)
  - Mempertahankan instruksi terakhir yang telah di fetch

Urutan Proses Fetch

1. Alamat instruksi selajutnya pada PC
2. Alamat pada MAR diletakkan pada bus alamat
3. Control unit memberi perintah READ
4. Hasil (data dari memori) berada dalam bus data
5. Data dari bus data di-copy ke MBR
6. PC akan bertambah 1 (paralel dngan proses fetch data dari memori
7. Data (instruksi) dipindahkan dari MBR ke IR
8. MBR sekarang siap untuk data pada proses fetch selanjutnya

Urutan Proses Fetch (dalam bentuk simbol)

t1: MAR <- (PC)
t2: MBR <- (memory)
PC <- (PC) +1
t3: IR <- (MBR)
(tx = Unit waktu /clock cycle)

atau

t1: MAR <- (PC)
t2: MBR <- (memory)
t3: PC <- (PC) +1
IR <- (MBR)

Aturan Pengelompokan Untuk Siklus Clock

Setiap urutan harus diikuti
MAR <- (PC) harus mendahului MBR <- (memory)

Konflik haris dihindarkan

- Tidak boleh membaca dan menulis register yang sama dalam waktu yang sama
- MBR <- (memory) da IR <- (MBR) tidak boleh dalam siklus yang sama

Begitu juga :

- PC <- (PC) +1 melibatkan suatu penambahan
- Menggunakan ALU
- Membutuhkan tambahan operasi mikro yang lain

Siklus In-direct

1. MAR <- ($IR_{address}$) - field alamat IR
2. MBR <- (memory)
3. $IR_{address}$ <- ($MBR_{address}$)
4. MBR mengandung alamat
5. IR sekarang dalam state yang sama seperti pengalamatan langsung yang digunakaan

Siklus Interupsi

t1: MBR <-(PC)
t2: MAR <- alamat-yang disimpan
PC <- alamat-rutin
t3: memory <- (MBR)

- Di atas adalah perintah minimum :
- Perlu tambahan operasi mikro lagi dalam mengambil alamat yang disimpan.
- Isi yang disimpan dilakukan oleh rutin interrupt handler, bukan operasi mikro.

Siklus Eksekusi (ADD)

- Akan berbeda untuk setiap instruksi
- Misalnya ADD R1,X ← akan menambahkan sejumlah bilangan pada alamat X ke Register 1 , hasilnya di R1
  t1: MAR <- ($IR_{address}$)
  t2: MBR <- (memory)
  t3: R1 <- R1 + (MBR)
- Ini contoh untuk operasi mikro yang tidak overlap.

Siklus Eksekusi (ISZ)

ISZ X - increment dan skip bila zero
t1: MAR <- ($IR_{address}$)
t2: MBR <- (memory)
t3: MBR <- (MBR) + 1
t4: memory <- (MBR)
if (MBR) == 0 then PC <- (PC) + 1
Catatan :
Ini terjadi dalam operasi mikro tunggal
Operasi mikro selesai pada t4

Siklus Eksekusi (BSA)

BSA X - Branch dan save address
Alamat instruksi akan mengikuti BSA , untuk disimpan di X
Eksekusi akan berlanjut dari X+1

t1: MAR <- ($IR_{address}$)
MBR <- (PC)
t2: PC <- ($IR_{address}$)
memory <- (MBR)
t3: PC <- (PC) + 1

Kebutuhan Fungsional

1. Definisikan elemen dasar prosesor
2. Deskripsikan operasi mikro yang harus dilakukan prosesor
3. Menentukan fungsi control unit yang harus dilakukan prosesor

1. Elemen dasar Prosesor

- ALU
- Registers
- Internal data path
- External data path
- Control Unit

2. Tipe Operasi Mikro

- Transfer data antar register
- Transfer data dari register ke eksternal
- Transfer data dari eksternal ke register
- Operasi Aritmatik atau logical

3. fungsi control unit

a. Sequencing (mengurutkan operasi)
Membuat sejumlah operasi CPU akan melalui urutan operasi mikro tertentu

b. Mengeksekusi
Membuat kinerja setiap operasi mikro selesai dengan menggunakan sinyal kontrol tertentu

c. Jenis sinyal kontrol

Clock

- Satu instruksi operasi mikro (atau satu set operasi mikro yang paralel) per siklus clock

Instruction register

- Op-code untuk setiap instruksi
- Menentukan intruksi mikro mana yang akan dilakukan

Flags

- Menunjukkan status CPU
- Menunjukkan hasil dari operasi sebelumnya

Yang ada pada bus kendali

- Interupsi
- Sinyal Acknowledgement

Yang ada pada Output

Dalam CPU

- Untuk pergerakan data
- Untuk mengaktifkan fungsi tertentu

Yang melalui bus kendali

- ke memori
- ke modul I/O

Contoh Urutan Sinyal Kontrol Proses Fetch

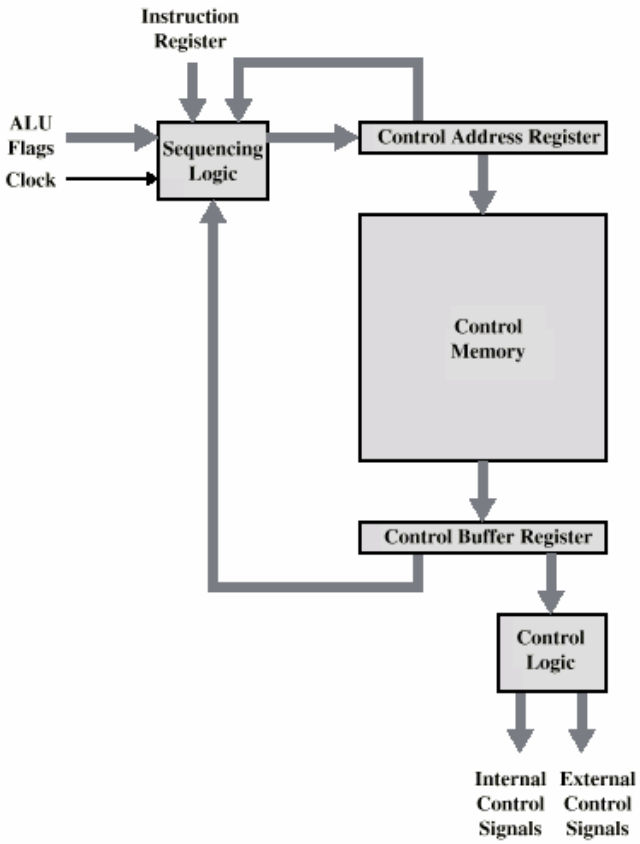
MAR <- (PC)

Control unit mengaktifkan sinya untuk membuka gerbang antara PC dan MAR

MBR <- (memory)

- Membuka gerbang anatara MAR dan bus alamat
- Sinyal kontrol pembacaan memori
- Membuka gerbang antara data bus dan MBR

Sinyal-sinyal pada Control Unit



SISTEM INPUT DAN OUTPUT

Permasalahan dalam I/O

- Amat banyak variasi jenis peripheral :
  - Menangani data dengan cara yang berbeda
  - Dengan kecepatan yang berbeda
  - Dengan format yang berbeda
  - Kesemuanya lebih lambat dari CPU dan RAM
- Memerlukan modul I/O

Modul I/O

Interface dengan CPU dan Memori
Interface ke satu atau lebih peripheral

Device Eksternal

Pembacaan disisi manusia :
Screen, printer, keyboard

Pembacaan disisi mesin :
Monitoring and control

Komunikasi :
Modem
Network Interface Card (NIC)

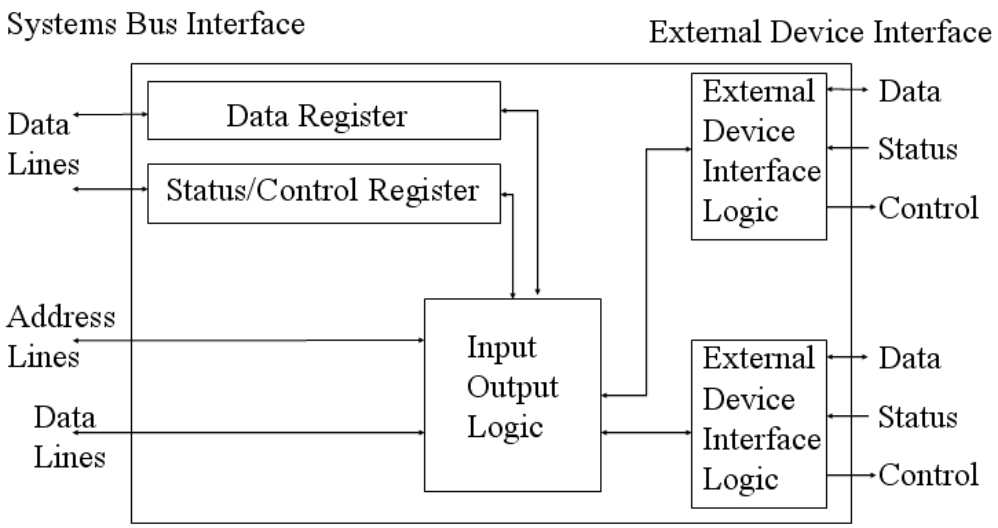
Fungsi Modul I/O

1. Control & Timing
2. Komunikasi CPU
3. Device untuk komunikasi
4. Data Buffering
5. Deteksi Error

Langkah-langkah penanganan I/O

1. CPU mengecek status modul I/O device
2. I/O module mengirimkan statusnya
3. Jika ready, CPU meminta transfer data
4. I/O modul mengambil data dari device
5. I/O modul transfer data ke CPU dalam variasi output yang diinginkan.

Diagram Modul I/O



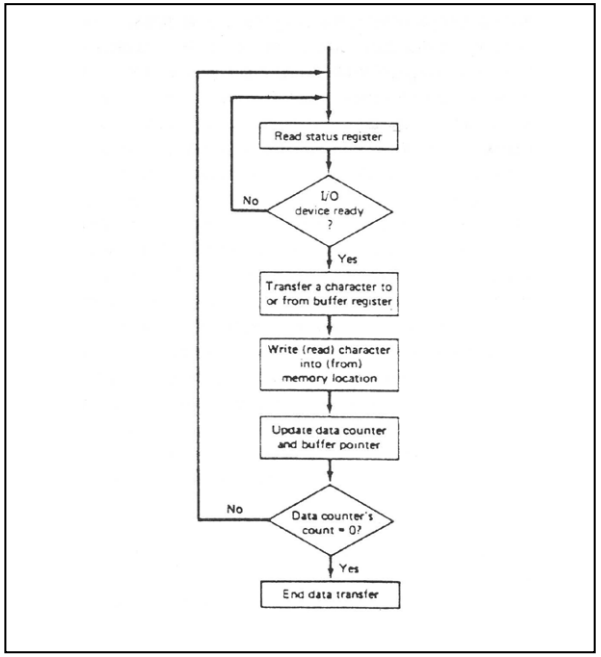
Metode Pengaksesan Sistem I/O :

1. **Memori Mapped I/O**
   Piranti I/O dihubungkan sebagai lokasi memori virtual sehingga port I/O tergantung memori utama.
   Karakteristik :
   - Port I/O → menempati bagian tertentu pada bus Alamat, diakses seolah-olah lokasi memori.
   - Piranti input → menjadi bagian dari memori yang memberi data ke bus data.
   - Piranti output → menjadi bagian dari memori yang memiliki data di dalamnya.

2. **I/O Mapped I/O**
   Piranti I/O dihubgungkan sebagai lokasi terpisah dari lokasi memori, sehingga port I/O tidak tergantung pada keadaan memori utama.
   Karakteristik :
   - Port I/O → tidak tergantung memori utama
   - Transfer informasi → dibawah kendali sinyal kontrol dengan instruksi input dan output.
   - Operasi I/O → tergantung sinyal kendali dari CPU.
   - Jenis instruksi :
     - Instruksi I/O → mengaktifkan baris kendali read/write pada port I/O.
     - Instruksi memory → mengaktifkan baris kendali read/write pada memori.
   - Ruang memori dan ruang alamat I/O bisa saja memiliki alamat yang sama.
   - Lebih cepat dan efisien karena penggunaan alamat yang terpisah, namun memiliki keterbatasan jumlah instruksi yang dapat digunakan untuk operasi I/O.

Metode Operasi Sistem I/O :

1. **I/O Terprogram**
   → CPU mengendalikan operasi I/O secara keseluruhan dengan menjalankan serangkaian instruksi I/O dengan suatu program tertentu.



Gambar : Flowchart I/O Program

→ Karakteristik :
- Terdapat program untuk : **memulai** - **mengarahkan** - **menghentikan** operasi I/O.
- Membutuhkan perangkat keras register :
  - Register status → status piranti I/O dan data yang akan dikirim
  - Register buffer → untuk menyimpan data sementara sampai CPU / piranti I/O siap.

- Register pointer buffer → menunjuk ke lokasi memori dimana sebuah karakter harus ditulis atau dibaca.
- Register counter data → tempat penyimpanan jumlah karakter dan akan berkurang nilainya jika karakter berhasil ditransfer.

- Perlu waktu proses yang menyita waktu pemanfaatan CPU.

2. I/O Interupsi (Demand Driven)

→ CPU akan bereaksi ketika suatu piranti mengeluarkan permintaan untuk pelayanan.

→ Karakteristik :

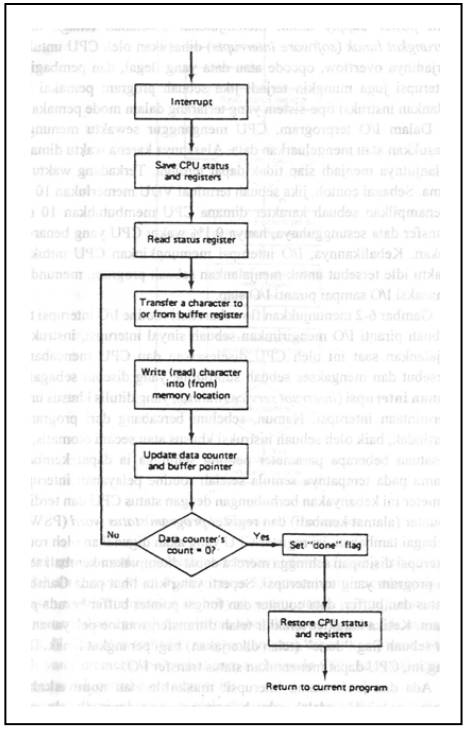
- lebih efisien dalam pemanfaatan CPU → tidak harus terus menerus menguji status dari piranti.
- Menunggu interupsi dari piranti I/O.

Jenis interupsi :

1. Interupsi hardware : timer, memori, powersupply dan lain-lain
2. Interupsi Software : overflow, ilegal opcode/data, divide by zero

Sifat interupsi :

1. Maskable : interupsi yang dapat di-disable untuk sementara dengan instruksi *Disable interupsi* khusus.
2. Non maskable : interupsi yang tidak dapat di-disable dengan instruksi perangkat lunak.



Gambar : Flowchart I/O Interupsi

- Karena banyaknya piranti yang meminta interupsi, menuntut CPU untuk dapat menentukan prioritas layanan. Ada 2 cara metode pemilihan prioritas layanan :
  1. **pooling / *Pooled interrupt*** : piranti-piranti dihubungkan dengan suatu gerbang OR ke kendali Interupsi (INTR=INTERRUPT REQUEST). CPU akan menguji bit status piranti, jika permintaan interupsi ditemukan, rutin pooling akan bercabang ke lokasi memori tempat rutin pelayan interupsi berada.
  2. **vector interrupt** : piranti yang berinterupsi diidentifikasi secara langsung dan dihubungkan dengan rutin pelayanan interupsinya. Caranya :

a. Piranti akan dihubungkan dengan baris intruksi INTR yang memang secara organisasi di set secara berlainan.
b. CPU men-set baris INTA (INTerrupt Acknowledgement) untuk mengarahkan masing-masing pinterupsi tersebut ke baris instruksi layanannya.
c. Piranti menerapkan interrupt vektor pada bus.
d. CPU bercabang ke alamat memori yang ditunjuk interrupt vektro tersebut.

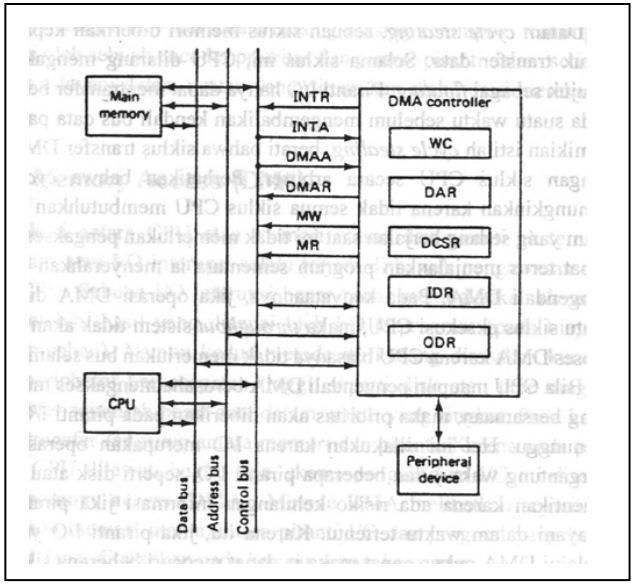
3. Direct Memory Access (DMA)

→ Metode transfer data secara langsung antara memori dan piranti tanpa pengawasan dan pengendalian CPU.

→ Hanya dapat dilakukan pada piranti I/O berkecepatan tinggi dan mampu mentransfer data besar dalam waktu singkat seperti disk.

→ Metode :

1. CPU hanya menginisialisasi saluran DMA.
2. DMA controller kemudian mengendalikan sepenuhnya transfer data, dengan 2 cara :
   - **skema transfer blok DMA dual port** : CPU dan DMA controller mengakses memori utama memlalui MAR dan MBR, pada memori utama jenis dual port (port 1 → CPU, port 2 → DMA controller)
   - **skema transfer DMA Cycle Stealing** : Memakai memori utama jenis port tunggal, sehingga CPU dan piranti I/O akan beradu cepat mendapatkan tempat layananan pada basis asynchronous independen



Gambar : Pengendali DMA

Transfer Data pada Sistem I/O :

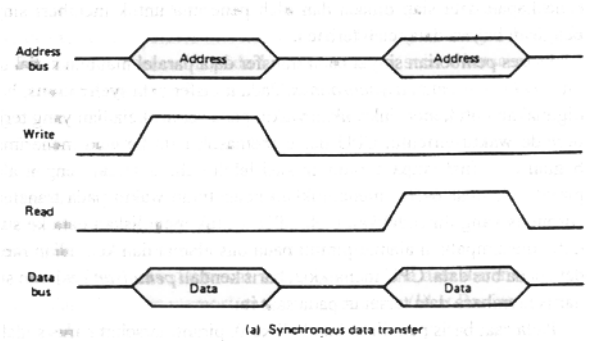
1. Format Transfer

a. **Paralel** : semua bit pada karakter (word dengan panjang tertentu) dikirim secara bersamaan dalam batas waktu transmisi tertentu.
b. **Serial** : data dikirim secara berurutan dalam satu bariskomunikasi tungga, sehingga antara pengirim dan penerima harus membagi batas waktu pengiriman karakter menjadi beberapa sub interval pengiriman/penerimaan.

Transfer paralel lebih cepat karena memiliki saluran transmisi yang banyak, namun tidak bisa diterapkan pada jarak yang terlalu panjang, karena dapat terjadi interferensi antar saluran.

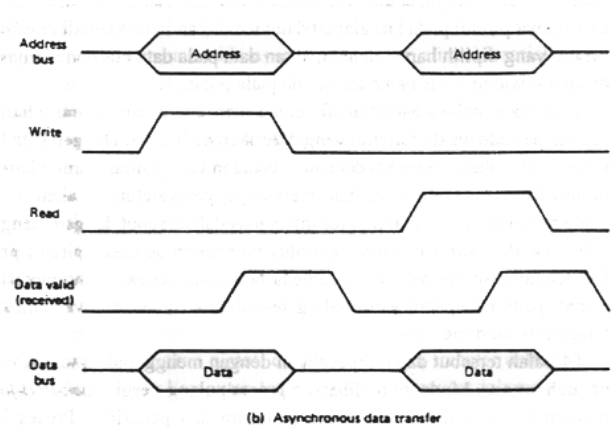
2. Mode Transfer

a. **Synchronous** : Baris kendali dipakai untuk emsinkronkan waktu pada semua kejadian yang terjadi selama periode tertentu. Setiap piranti harus bereaksi dengan kecepatan yang sama pada periode yang diberikan CPU. Namun karena kecepatan piranti I/O yang bervariasi sedang data dikirim secara serial dan bergantian dalam periode yang sudah ditetapkan, maka kecepatan transfer di set pada piranti I/O dengan kecepatan terendah.



Gambar : Transfer Synchronous

b. **Asynchronous** : Proses back-and-forth (pulang-pergi) dalam meneruskan sinyal kendali dari pengirim ke penerima. Tekniknya bernama handshaking dengan melakukan terlebih dahulu pengecekan validitas data yang akan transfer. Kelemahan : perlu banyak kendali. Kelebihan : memungkinkan penggunaan piranti I/O yang memiliki berbagai variasi kecepatan dalam sistem yang sama.



Gambar : Transfer Asynchronous

Interfacing Piranti I/O

→ **Defenisi :** Suatu alat yang digunakan untuk menghubungkan suatu piranti dengan CPU melalui BUS.
→ **Fungsi Umum :** Mensinkronkan data transfer antara CPU dan piranti I/O
→ **Fungsi Detail :**

- Penyedia status piranti I/O bagi CPU
- Memiliki kemampuan interupsi / DMA
- Mampu me-transfer instruksi CPU ke piranti
- Mampu berfungsi sebagai buufer storage data transfer.
- Mampu melakukan pengujian kesamaan data.
- Mampu men-decode dan mgng-encode data
- Memiliki fasilitas khusus :
    - Konversi data paralel-ke-serial dan sebaliknya.
    - Encoding karakter ketik ke VDU.
    - Encoding karakter ketik tertentu : F1, F2, BACKSPC, DELETE dan sebagainya.
- Menyediakan sinyal status operasi.

→ **Komponen Utama Interface Piranti I/O :**

1. Device dependent → bagian yang melayani piranti
2. Device independent → bagian yang menghubungkan unit interface ke bus sistem.

→ **Struktur Interface Piranti I/O :**

1. Register
   - kendali (CR) → mencatat instruksi dan informasi dalam piranti.
   - Status (SR) → mencatat status piranti dan mengeluarkan pesan kesalahan
   - Data input (IDR) dan data output (ODR) → sebagai buffer data untuk operasi input dan output.
2. Bus
   - Receiver → menangani data input
   - Transciever → sirkuit bi-directional data menangani input maupun output
   - driver/buffer bus → sirkuit tri-state yang menyimpan informasi bus.

Sistem Prosesor I/O :

→ **Defenisi :** General Purpose komputer yang berisi sejumlah saluran DMA, CPU tersendiri, kumpulan intruksi dan menjalankannya secara paralel pada CPU utama.

→ **Fungsi :** sebagai piranti front end yang menangani setiap aspek I/O, dan menyediakan pengendali I/O khusus yang disebut I/O Channel.

I/O Channel

Defenisi : Pengendali yang melakukan operasi I/O dalam mode DMA pada sistem bus yang mengatur komunikasi dengan memori utama pada piranti-piranti I/O melalui bus I/O → dapat melakukan deteksi dan perbaikan kesalahan serta beroperasi pada basis cycle stealing.

Macam layanan :

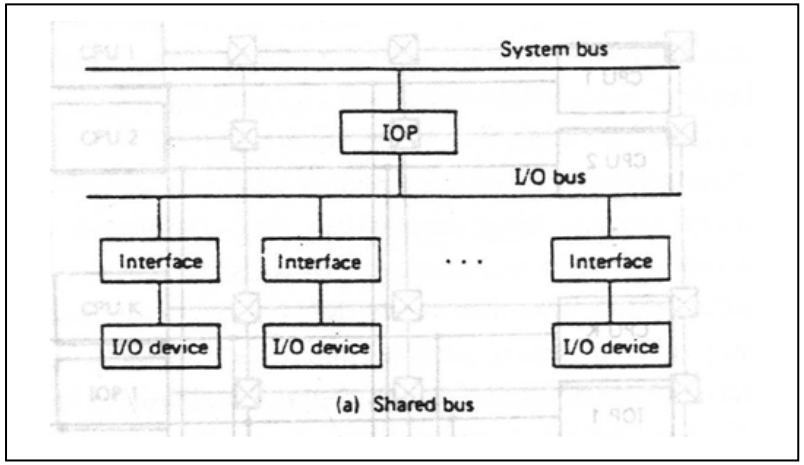
- Saluran multiplexer → menghubungkan piranti berkecepatan rendah / sedang, lalu mengoperasikannya secara bersamaan dengan multiplexing.
- Saluran selektor → menghubungkan piranti I/O berkecepatan tinggi tanpa multiplexing.
- Saluran multiplexer blok → kombinasi layanan di atas.

→ **Instruksi IOP :**

1. Instruksi transfer data → untuk Input (pembacaan), output (penulisan) dan untuk membaca informasi status.
2. Instruksi general purpose → instruksi aritmatika, logika, percabangan (konversi, prioritas operasi, evaluasi alamat, jump instruksi)
3. Instrusksi kendali → instruksi untuk menangani fungsi piranti I/O khusus yang tidak terlibat dalam transfer data. Contoh : pmemindahkan head R/W untuk menentukan lokasi track dan record pada disk, mencetak pindah baris pada printer, dan-lain-lain.

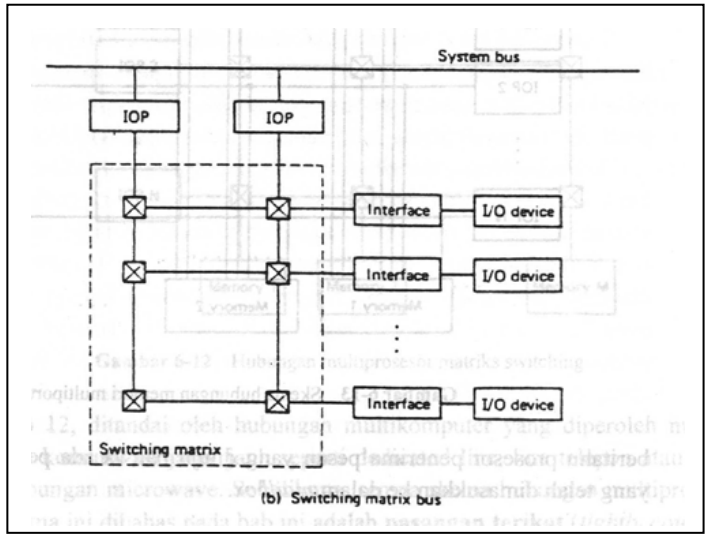
→ **Mode IOP :**

1. **Single shared Bus** → setiap IOP mengendalikan sejumlah piranti I/O tertentu yang tetap



Gambar : Single Share Bus

2. **Switching matriks bus** → pensaklaran yang memungkinkan transfer bersamaan antara I/O dan modul memori.



Gambar : Switching Matriks Bus

→ **Format Instruksi IOP :**

1. Field opcode → representasi jenis operasi
2. field alamat memori → alamat awal blok memori yang digunakan untuk transfer.
3. field word count → jumlah word yang harus ditransfer (panjang blok memori).
4. field kendali → untuk fungsi-fungsi piranti I/O khusus.
5. Field status → untuk tujuan komunikasi dan pencatatan.

TEKNOLOGI SISTEM MEMORI

I. Karakteristik memori :

1.kapasitas
2. satuan transfer
3. metode akses
4. kinerja
5. tipe fisik
6. karakteristik fisik

Kapasitas

Kapasitas dinyatakan dalam byte (1 byte 8 bit) atau word. Panjang word yang umum adalah 8, 16, dan 32 bit.

Satuan Transfer

Satuan transfer sama dengan jumlah saluran data yang masuk ke dan keluar dan modul memori. Tiga konsep dalam satuan transfer:

- Word. Ukuran word biasanya sama dengan jumlah bit yang digunakan untuk representasi bilangan dan panjang instruksi.
- Addressable Units. Pada sejumlah sistem, Addressable Unit adalah word. Hubungan antara panjang A suatu alamat dan jumlah N addressable unit adalah $_2A = N$.
- Unit of Transfer Adalah jumlah bit yang dibaca atau yang dituliskan ke dalam memori pada suatu saat.

MetodeAkses

Terdapat empat jenis metode:

- SequentialAccess. Memori diorganisasikan menjadi unit-unit data yang disebut record.
- Direct Access. Direct Access meliputi shared read/write mechanism. Setiap blok dan record memiliki alamat-alamat yang unik berdasarkan lokasi fisik.
- Random Access. Waktu untuk mengakses lokasi tertentu tidak tergantung pada urutan akses sebelumnya dan bersifat konstan
- Associative. Sebuah word dicari berdasarkan pada isinya dan bukan berdasar pada alamat.

Metode Sequential Access dan Direct access, biasanya dipakai pada Memori pembantu. Metotle Random Access dan Associative dipakai dalam Memori Utama.

Kinerja

Pada memori utama, terdapat tiga buah parameter unjuk kerja:

- Access Time. Bagi RAM, access time merupakan waktu yang dibutuhkan untuk melakukan operasi baca atau tulis. Bagi non RAM, access time adalah waktu yang dibutuhkan untuk melakukan mekanisme baca tulis pada lokasi tertentu.
- Memory Cycle Time. Terdiri dan access time ditambah dengan waktu tam bahan yang diperlukan transient agar hilang pada saluran signal atau untuk menghasilkan kembali data bila data ini dibaca secara destruktif.
- Transfer Rate. Transfer rate adalah kecepatan data agar dapat ditransfer ke unit memori atau ditransfer dan unit

memori. Pada RAM, transfer rate = 1/(waktu siklus). Bagi non RAM terdapat hubungan:

$$TN = TA + \frac{N}{R}$$

Keterangan :

TN = Waktu rata-rata untuk membaca atau menulis N bit.
TA = Waktus access rata-rata.
N = Jumlah bit.
R = Kecepatan transfer, dalam bit per detik (bps).

Tipe Fisik

Memori Utama dikemas dalam sebuah Chip IC. Dua jenis yang umum digunakan saat ini adalah
memori semikonduktor yang memakal teknologi LSI dan VSLI.

Karakteristik Fisik

- ✖ Memori volatile, informasi akan hilang apabila daya listrik dimatikan.
- ✖ Memori non volatile, informasi tetap akan tersimpan meskipun daya listrik dimatikan.

II. Klasifikasi memori :

A. memori utama : internal(RAM,DRAM,SDRAM) dan ekternal(ROM, PROM, EPROM, EEPROM, Cache)
B. memori pembantu (disk magnetik,pita magnetik, floppy disk, drum magnetik, optical disk

A. MEMORI UTAMA

Memori Semi Konduktor Random Access

Tipe Memori	Kategori	Penghapusan	Mekanisme penulisan	Volatilitas
RAM	Read-write Read-only	Electrically byte level	Electrically	Volatile
ROM	Read only memory	Tidak mungkin	Mask	Non Volatile
PROM				
EPROM	Read mostly memory	Sinar Ultra Violet		
Flash Memory		Electrically block level		
EEPROM		Electrically byte level		

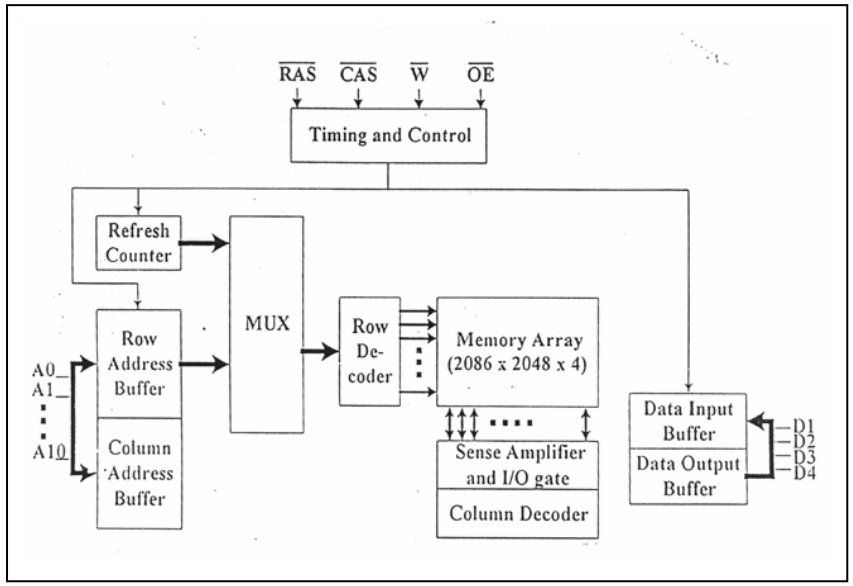
Menurut mekanisme yang digunakan untuk menyimpan dan memanggil data, memori utama dapat diklasifikasikan atas Random Access Memory dan Content Addressable Memory.

1. RAM (Random Access Memori)

→ RAM diakses melalui alamat, semua lokasi yang dapat dialamati dapat diakses secara acak (random) dan membutuhkan waktu akses yang sama tanpa tergantung pada lokasi fisiknya didalam memori.

→ Terdapat dua jenis RAM, statik dan dinamik :

- ✖ RAM dinamik tersusun oleh sel-sel yang. menyimpan data sebagai muatan listrik pada kapasitor.
- ✖ RAM statik menyimpan nilai-nilai biner dengan menggunakan konfigurasi gerbang logika flip-flop

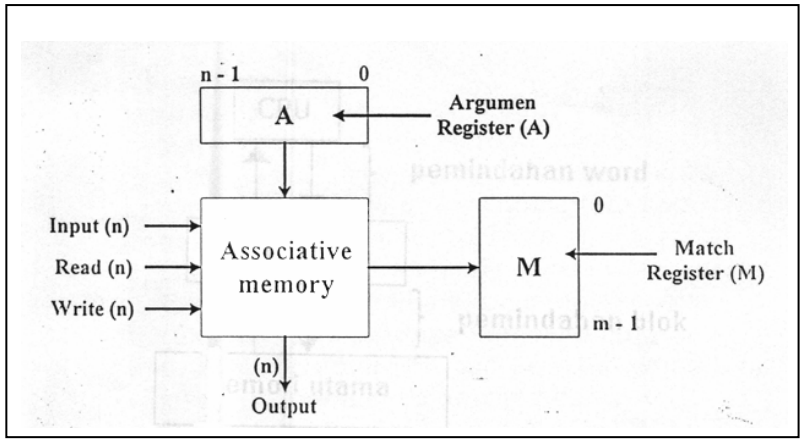


Gambar : Organisasi DRAM 16MB

2. CAM (content Address Memory)

→ pada CAM, memori diakses berdasarkan isi bukan alamat. pencarian data dilakukan secara simultan dan paralel dengan basis isi memori.

→ CAM disebut juga memori asosiatif.

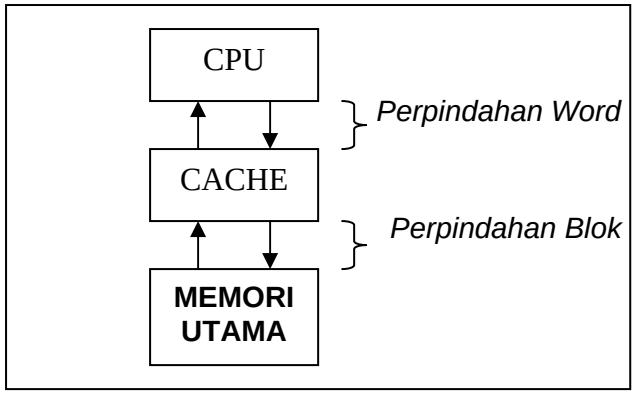


Gambar : Diagram awal memori asosiatif

3. Memory cache

→ **Latar belakang**

Memori utama yang digunakan sistem komputer pada awalnya dirasakan masih lambat kerjanya dibandingkan dengan kerja CPU, sehingga perlu dibuat sebuah memori yang dapat membantu kerja memori utama tersebut. sebagai perbandingan waktu akses memori cache lebih cepat 5-10 kali dibandingkan memori utama.



Gambar : Memori cache dan memori utama

→ **Prinsip kerja Cache**

- ✖ Cache berisi salinan sebagian isi memori utama. Pada saat CPU membaca sebuah word memory, maka dilakukan pemeriksaan untuk mengetahui apakah word tersebut berada di cache.
- ✖ Jika word memori terdapat di cache, maka akan dikirimkan ke CPU yang dikenal sebagai **proses HIT**.
- ✖ Sedangkan bila tidak ada, maka blok memori utarna yang terdiri dan sejumlah word tetap akan diletakkan/dikopikan di cache yang dikenal sebagai **proses MISS** dan selanjutnya dikirimkan ke CPU.

→ **Elemen-elemen rancangan cache**

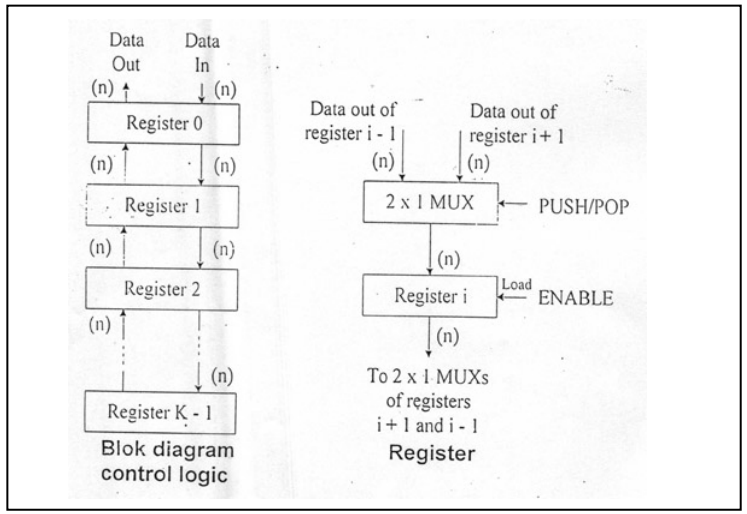
- **Ukuran cache** : Ukuran cache disesuaikan kebutuhannya dalam membantu kerja memori utama. Semakin besar ukuran cache, maka semakin besar jumlah gerbang (gate) yang terdapat pada pengalamatan cache, akibatnya adalah cache yang berukuran besar cenderung untuk lebih lambat dibanding dengan cache berukuran kecil. Berdasarkan penelitian ukuran cache antara 1K sampai 512K word akan lebih optimum dalam membantu kerja memori utama.
- **Fungsi pemetaan (mapping)** : Saluran cache lebih sedikit jumlahnya dibandingkan saluran blok memori utama sehingga perlu algoritma untuk pemetaan blok-blok memori ke dalam saluran cache dan perlu juga alat untuk menentukan blok memori utama yang sedang memakai saluran cache. Pemilihan fungsi pemetaan seperti pemetaan langsung, asosiatif dan asosiatif set akan menentukan bentuk organisasi cache.
  - **Pemetaan langsung :** Teknik yang paling sederhana, yaitu memetakkan masing-masing blok memori utama hanya ke sebuah saluran cache saja. Fungsi pemetaan mudah diimplementasikan dengan menggunakan alamat. Cache diakses dengan menggunakan alamat memori utama dianggap terdiri tiga field yaitu **tag**, **line**, dan **word**. Kekurangannya yang utama adalah terdapat lokasi cache yang tetap bagi sembarang blok-blok yang diketahui. Dengan demikian, apabila suatu program berulang-ulang melakukan word referensi dan dua blok yang berbeda memetakan ke saluran yang sama, maka blok-blok itu secara terus menerus akan di-swap ke daam cache, akibatnya hit ratio.-nya akan rendah.
  - **Pemetaan asosiatif :** Mengatasi kekurangan pemetaan langsung dengan cara mengizinkan setiap blok memori utama untuk dimuatkan ke sembarang saluran cache. Dalam hal mi, cache control logic menginterpretasikan alamat memori hanya sebagai sebuah field tag dan field word. Field tag secara unik mengidentifikasi suatu blok memori utama. Untuk

menentukan apakah suatu blok berada di dalam cache, maka cache control logic harus secara simultan memeriksa setiap tag saluran yang sesuai. Dengan pemetaan asosiatif, terdapat fleksibilitas penggantian blok ketika sebuah blok di baca ke dalam cache. Kekurangan pemetaan ini adalah kompleksitas rangkaian yang diperlukan untuk menguji tag seluruh saluran cache secara paralel.

- **Algoritma penggantian** : Digunakan untuk menentukan blok mana yang harus dikeluarkan dari cache untuk menyiapkan tempat bagi blok baru. Ada 2 metode yaitu:
    - **Write-through :** Cache dan memori utama diupdate secara bersamaan waktunya. Keunggulannya salinan data di memori utama dan cache tetap, sedangkan kelemahannya pada proses “write” mernerlukan jumlah waktu sama dengan proses MISS.
    - **Write-back :** Melakukan update data di memori utama hanya pada saat word memori telah dimodifikasi dari cache. Keunggulannya proses update word cache tidak terbatas, sedangkan kelemahannya salinan data di memori utama tidak tetap /konsisten selama data termodifikasi benar-benar ada di mernoni utama.
- **Ukuran blok :** Blok-blok yang berukuran lebih besar mengurangi jumlah blok yang menempati cache. Setiap pengambilan blok menindih isi cache yang lama, maka sejumlah kecil blok akan menyebabkan data menjadi tertindih setelah blok itu diambil. Dengan meningkatnya ukuran blok, maka jarak setiap word tambahan menjadi lebih jauh dari word yang diminta, sehingga menjadi lebih kecil kemungkinannya untuk diperlukan dalam waktu dekat.

Implementasi Memori Utama

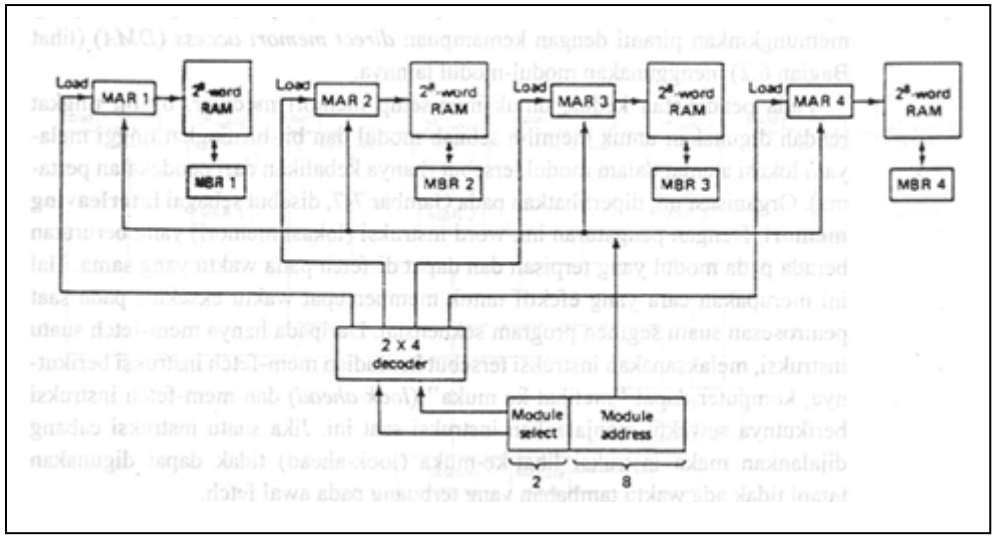
1. Memori Stack : Memori Stack merupakan struktur data tidak tetap yang kembali dan digunakan untuk menyimpan parameter yang dilalui alamat dalam subroutine *call* dan *return*, memanipulasi alamat serta operasi aritmatika.



Gambar : Memory Stack

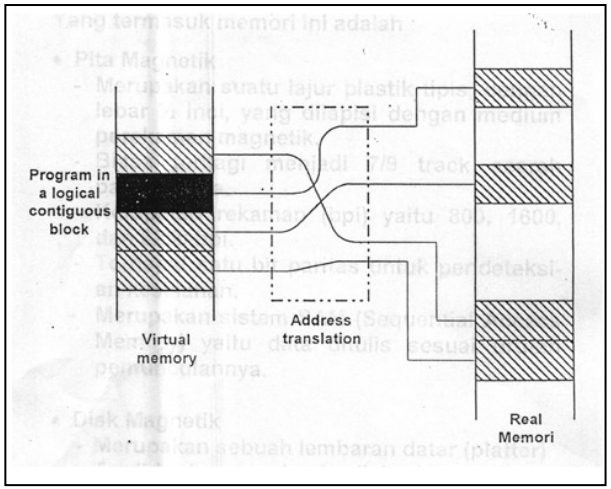
Pada gambar diatas, perintah PUSH berguna untuk memanipulasi data pada dan POP stack.

2. Memori Modular : Dalam sistem modular RAM dipisah menjadi modul-modul yang berbeda, yaitu MAR dan MBR. Penggunaan memori modular biasanya pada sistem dengan prosesor pipeline dan prosesor array.



Gambar : Memori Modular

3. Memori Virtual
prinsip dasar memori virtual adalah kemampuan untuk mengalamati ruang penyimpanan logikal yang secara fisik lebih besar daripada ruang penyimpanan riil.



Gambar : Memori Virtual

B. Memori pembantu (auxiliary memory)

- Bersifat non-volatile, yaitu jika tidak ada listrik, maka isi memori tidak hilang.
- Tidak mempengaruhi langsung fungsi CPU.

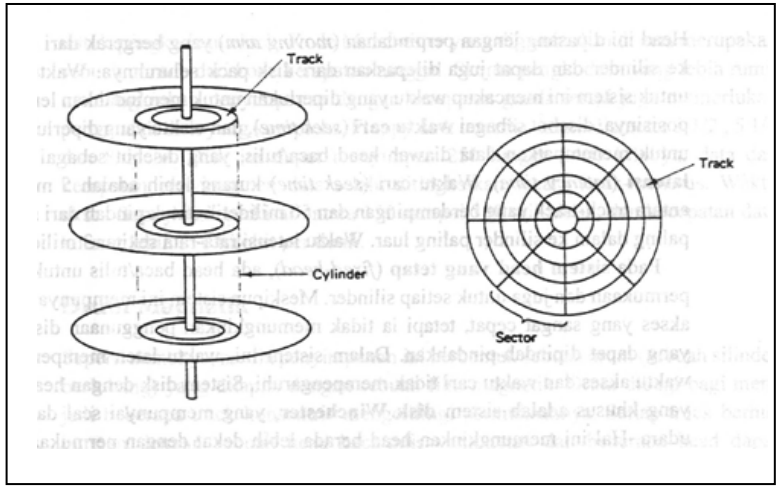
Yang termasuk memori ini adalah:

- Pita Magnetik :
  - Merupakan suatu lajur plastik tipis, lebar ½ inci, yang dilapisi dengan medium perekaman magnetik.
  - Biasa terbagi menjadi 7/9 track panjang pita.
  - Kerapatan rekaman (bpi) yaitu 800, 1600, dan 6250 bpi.
  - Terdapat satu bit paritas untuk pendeteksian kesalahan.

- Merupakan sistem SAM (Sequential Access Memory) yaitu data ditulis sesuai urutan pemunculannya.

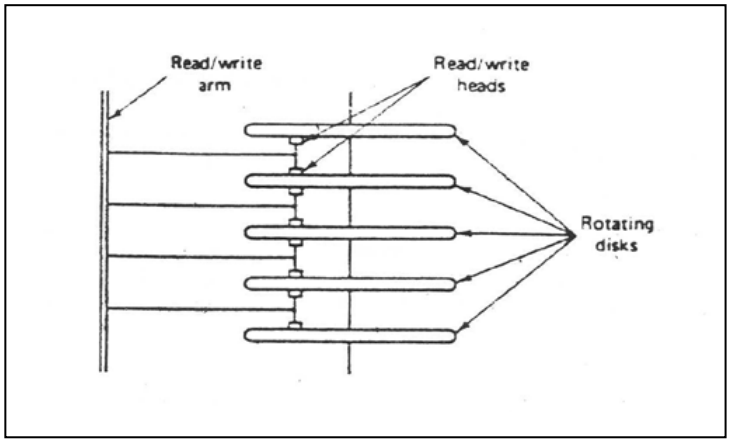
- Disk Magnetik :
  - Merupakan sebuah lembaran (platter)
  - Terdiri atas : sebuah kendali disk (interface), dan satu atau lebih disk (platter).
  - Proses penulisan ke disk yaitu disk drive akan menimbulkan kemagnetan pada titik di atas permukaan disk yang secara langsung di bawah head.
  - Proses pembacaan dan disk, head diatur agar dapat mendeteksi perubahan arah kemagnetan.
  - Terbagi secara logikal dikenal sebagai organisasi disk yaitu:

    Track : Sejumlah lingkaran yang konsentris
    Sektor : Pembagian perrnukaan disk secara belahan yang mempunyai ukuran yang sama.
    Silinder : Dibentuk oleh track-track yang berhubungan pada setiap permukaan



Gambar : Organisasi Disk Magnetik

- Floppy disk
  - Merupakan lembaran datar yang tipis dan fleksibel.
  - Hampir sama dengan harddisk, tetapi kap~sitas penyimpanan lebih rendah.



Gambar : Kumpulan Disk Magnetik

PIPELINING DAN RISC

Penemuan-penemuan Terkemuka dalam Perkembangan Komputer

- Konsep family komputer
  - IBM System/360 1964
  - DEC PDP-8
  - Dilihat berdasarkan implementasi arsitektur
- Micro-programmed control unit
  - Ide oleh Wilkes 1951
  - Produksi pertama oleh IBM S/360 1964
- Cache memory
  - IBM S/360 model 85 1969
- Solid State RAM
- Mikroprosesor
  - Intel 4004 1971
- Pipelining
  - Memperkenalkan paralelisme pada saat fetch siklus eksekusi
- Multiple prosesor
- Selanjutnya Reduced Instruction Set Computer

Reduced Instruction Set Computer (RISC)

- Fitur Utama
  - General purpose register dalam jumlah yang amat besar
  - Menggunakan teknologi compiler untuk mengoptimalisasikan penggunaan register.
  - instruction set yang sedikit dan sederhana
  - Pendekatan optimum dalam intruksi pipeline
  - Memimpin untuk
    - Set eksekusi yang besar
    - Lebih banyak mode pengalamatan

Perbandingan Jenis Prosesor

CISC			RISC		Superscalar	
IBM	DEC VAX	Intel	Motorola	MIPS	IBM	Intel
370/168	11/780	486	88000	R4000	RS/6000	80960
1973	1978	1989	1988	1991	1990	1989
No. of instruction						
208	303	235	51	94	184	62
Instruction size (octets)						
2-6	2-57	1-11	4	32	4	4 or 8
Addressing modes						
4	22	11	3	1	2	11
GP Registers						
16	16	8	32	32	32	23-256
Control memory (k bytes) (microprogramming)						
420	480	246	0	0	0	0

Karakteristik RISC :

- Satu instruksi per siklus
- Operasi register to register
- Mode pengalamatan yang sederhana
- format instruksi yang sederhana
- Desain Hardwired (tanpa microcode)
- Format instruksi yang Fix.
- Prose compile yang cepat

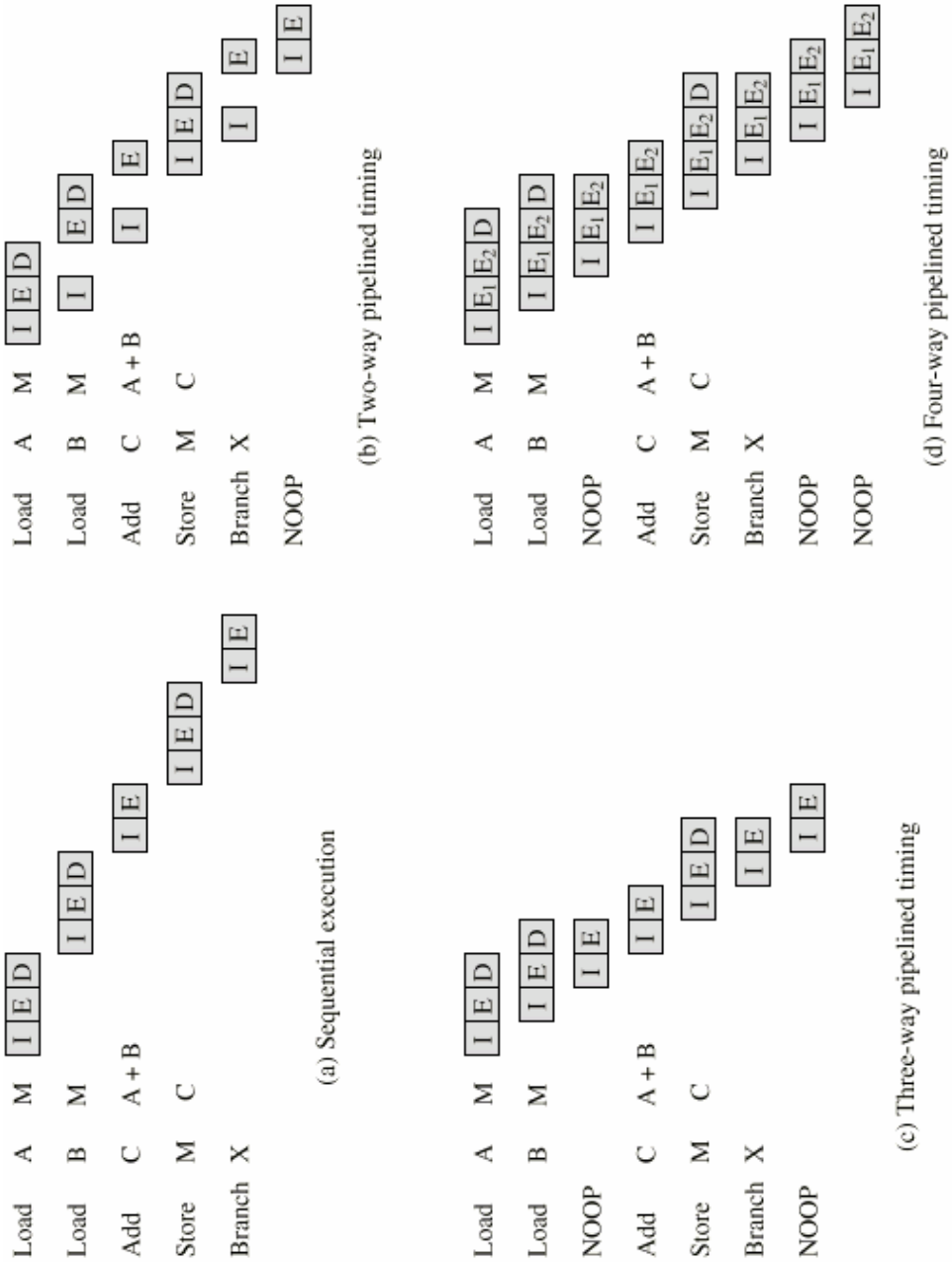
Pipelining pada RISC

- Terdapat berbagai macam instruksi pada register to register
- 2 fase eksekusi
  - I : Instruction fetch
  - E : Execute
    - Operasi ALU dengan register input dan output
- Untuk load dan store
  - I: Instruction fetch
  - E: Execute
    - Mengkalkulasi alamat memori
  - D: Memory
    - Operasi Register to memori atau memori to register

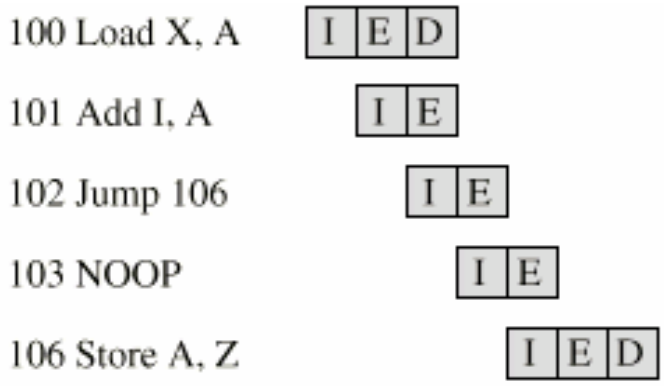
Optimalisasi Pipelining

- Percabangan yang dikenai delay
  - Tidak akan ada efeknya sampai suatu eksekusi intruksi selesai.
  - Instruksi percabangan ini akhirnya mengalami delay.

Efek Pipelining dalam RISC



Penggunaan Percabangan yang dikenai Delay



(a) Inserted NOOP

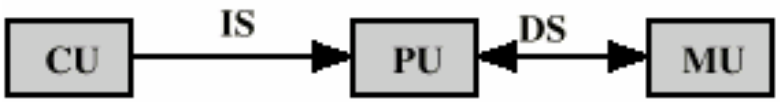
PROSESOR PARALEL

Pemrosesan Paralel

- Single instruction, single data stream - SISD
- Single instruction, multiple data stream - SIMD
- Multiple instruction, single data stream - MISD
- Multiple instruction, multiple data stream- MIMD

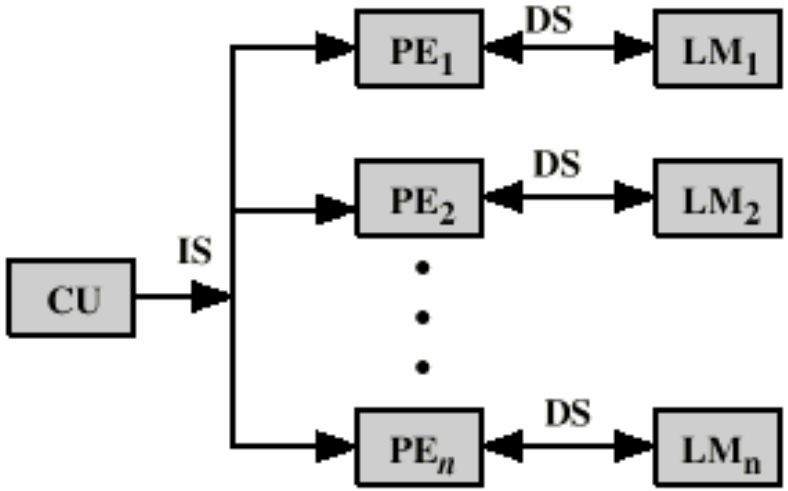
1. SISD

- Satu Prosesor
- Satu instruksi stream
- Data disimpan di satu memori
- Disebut Uni-processor



2. SIMD

- Intruksi mesin tunggal
- Ekskusi di kendalikan secara simultan
- Terdapat sejumlah elemen proses
- Berbasiskan langkah demi langkah
- Setiap elemen proses memiliki asosiasi dengan data di memori
- Setiap instruksi dieksekusi dalam set data yang berbeda oleh prosesor yang berbeda.
- Disebut juga Vector dan array processors



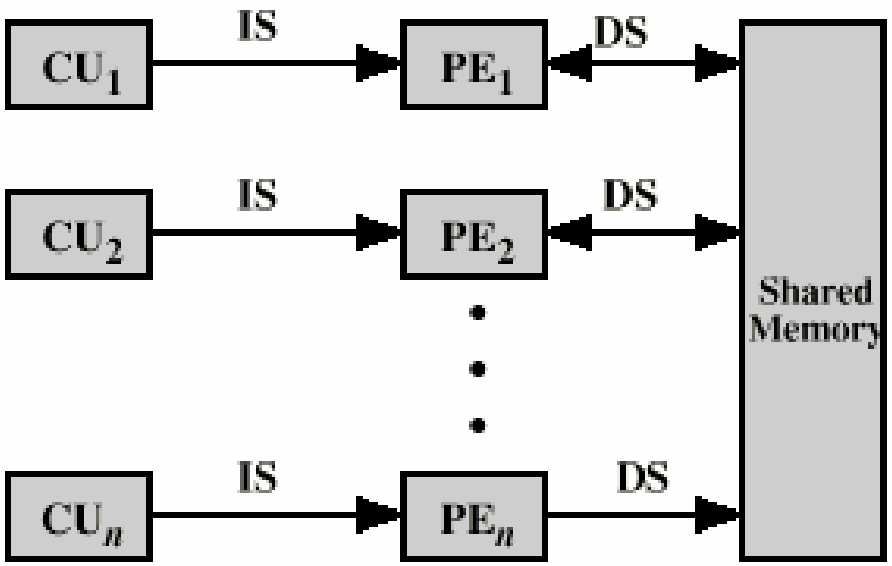
3. MISD

- Mengorganisir Urutan data
- Ditransmisikan oleh sjumlah set prosesor
- Setiap prosesor mengeksekusi urutan instruksi yang berbeda
- Belum dapat diimplementasikan dengan baik.

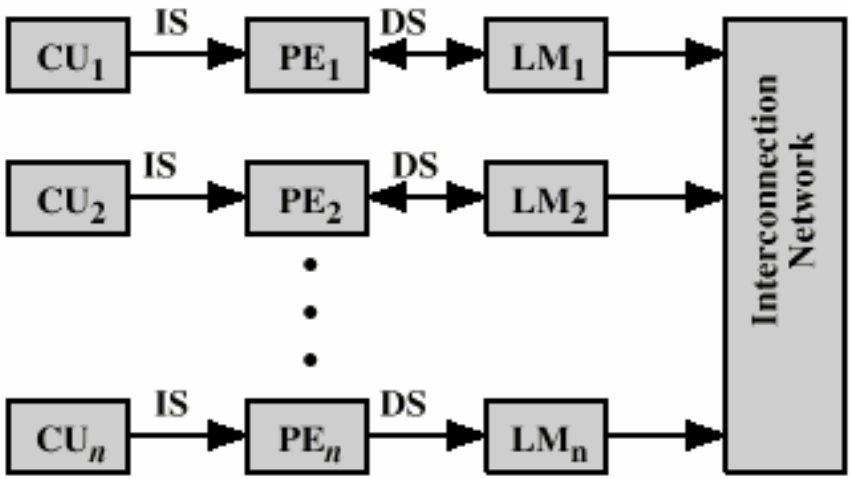
4. MIMD

- Terdiri dari sejumlah set prosesor
- Secara simultan mengeksekusi urutan instruksi yang berbeda
- Terdiri dari sejumlah set data yang berbeda
- Disebut juga SMP(Symmetric Multiprocessor), cluster dan NUMA (Nonuniform memory access) systems

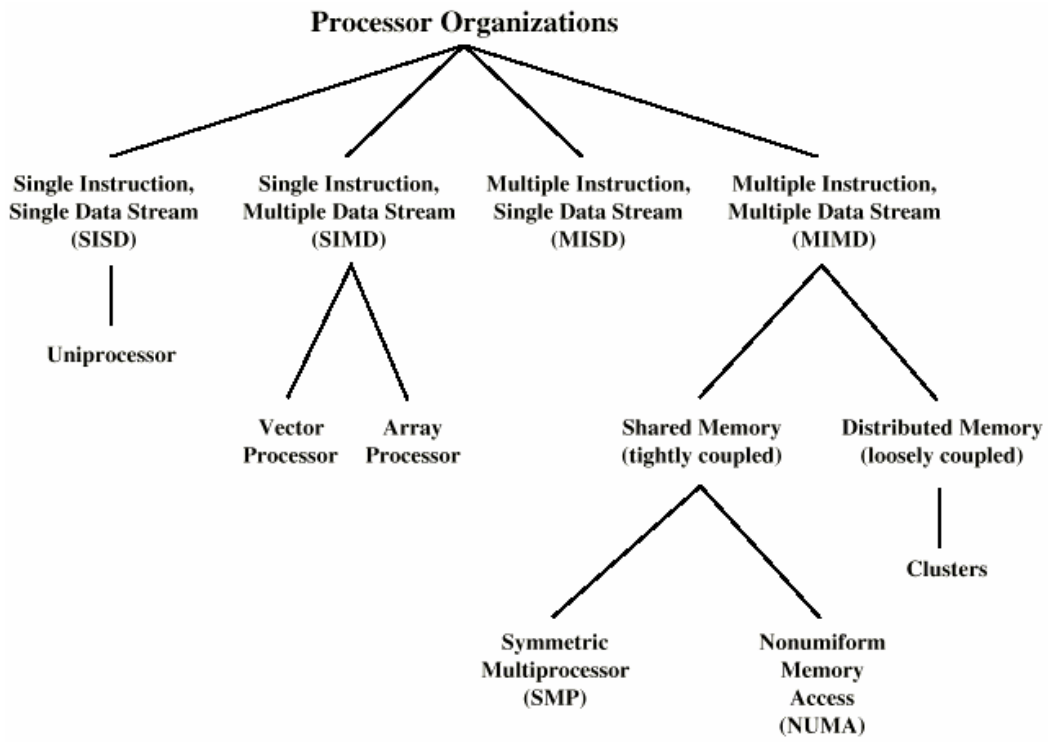
a. MIMD Share Memori



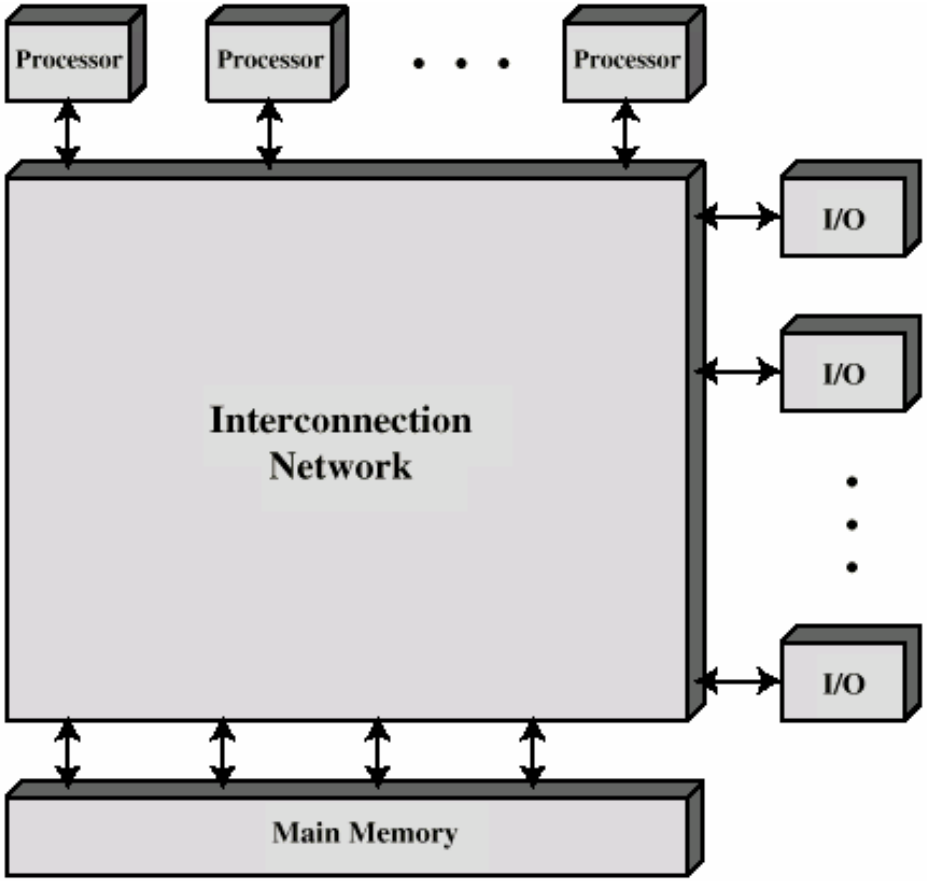
b. MIMD Distributed Memori



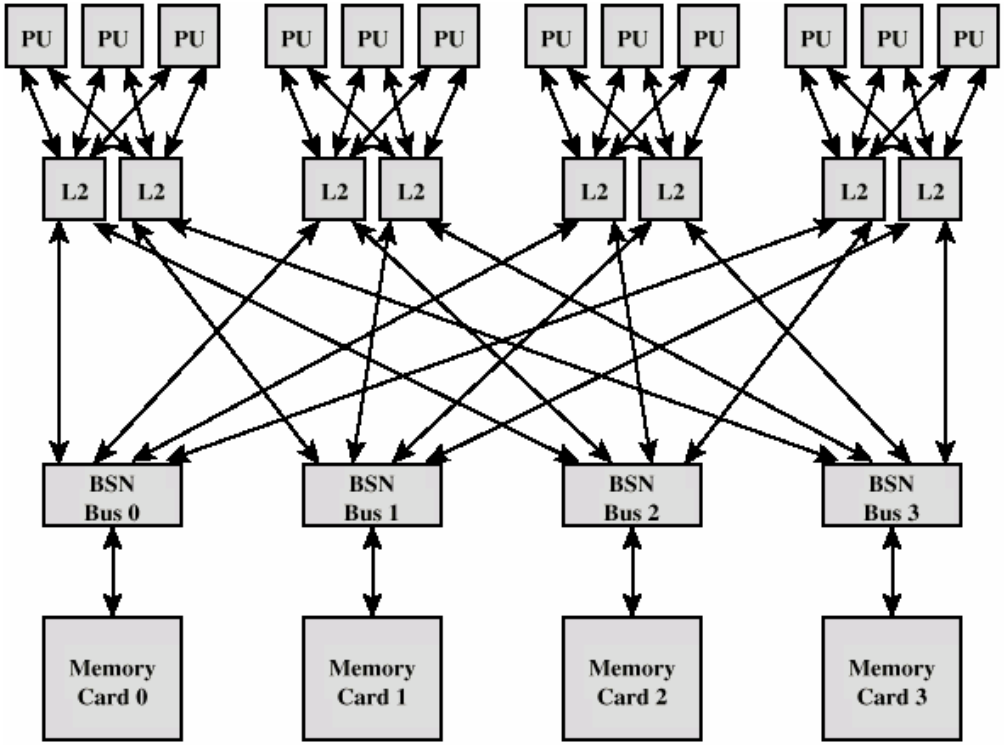
Taksonomi Arsitektur Paralel Prosesor



Blok diagram penerapan Prosesor Paralel



Contoh pada Mainframe IBM SMP S/390



- Processor unit (PU)
  - CISC microprocessor
  - Frequently used instructions hard wired
  - 64k L1 unified cache with 1 cycle access time
- L2 cache
  - 384k
- Bus switching network adapter (BSN)
  - Includes 2M of L3 cache
- Memory card
  - 8G per card

Document copyleft @ Mohammad Iqbal Skom, MMSI
Free distribution for educational purpose, 2004
With Bismillah TM